



An efficient certificateless public key cryptography with authorized equality test in IIoT

Rashad Elhabob¹ · Yanan Zhao¹ · Iva Sella¹ · Hu Xiong¹

Received: 11 February 2019 / Accepted: 11 June 2019 / Published online: 18 June 2019
© Springer-Verlag GmbH Germany, part of Springer Nature 2019

Abstract

Current time organizations have moved from the conventional industries to smart industries by embracing the approach of Industrial Internet of Things (IIoT), which has provided an avenue for the integration of smart devices and communication technologies. IIoT offers reduction of costs on services and technologies since it counts on the flexibility, availability and real-time data processing capabilities of the cloud server. And even with the untrusted nature of the cloud, the privacy and confidentiality of the data outsourced to the cloud has been able to be guaranteed. This has been achieved through encryption techniques apply to the data before outsourcing. This has made it feasible to have the data in the cloud secured. However, the limitation of data recovery due to it having the innate “all-or-nothing” decryption characteristic, results in a challenge in the search functionality for the encrypted data. In order to deal with this challenge, this paper nominates a Certificateless Public Key Cryptography with Authorized Equivalent Test (CL-PKC-AET). Whereby, a cloud server is given authority to actuate if two encryptions consist of the equivalent messages. Furthermore, the extra providence of a fine-grained testing capability to the cloud server makes the CL-PKC-AET scheme adaptable. We define our scheme basing on bilinear pairing and further advocate within the random oracle model its security, based on the Bilinear Diffie-Hellman assumption. Basing on comparison with existing work, our scheme proves to be more effective.

Keywords Industrial Internet of Things (IIoT) · Cloud · Certificateless public key cryptography with equality test · Authorization

1 Introduction

The present-day growths in hardware, software, and the cloud have resulted in the birth of the concept of Internet of Things (IoT) (Atzori et al. 2010; Singh et al. 2017; Kim and Lee 2018). IoT is an umbrella of a broad range of technologies and applications enabled by the connection of objects and devices. It has given a platform where smart gadget

devices like sensors, actuators, monitors can be accessed through the internet and be connected to other devices. These devices are utilized to offer identification, computing and communication capacities for individual targets. Recently, the Industrial Internet of Things (IIoT) has made very effective impacts particularly in the manufacturing sectors (Anagnostopoulos et al. 2016; Molano et al. 2018), where manufacturers and industrialist have embraced the concept of IIoT. Many have taken this opportunity to have their products and appliances connected to the IIoT. It has proved to be a rapidly progressing sector as it focuses on the use of cyber-physical systems to monitor the physical factory process and make decisions based on automated data. This has resulted to reduction in human errors but at the same time, an overall increase in efficiency and reduction in costs.

Having all these improved and intelligent devices and machines connected to the IIoT means a network that grows daily. Subsequently, the amounts of data that will be experienced in these networks are huge. Therefore, these networks are required to be intelligent and functional. Meaning the

✉ Hu Xiong
xionghu.uestc@gmail.com
Rashad Elhabob
rashaduestc@gmail.com
Yanan Zhao
zynbyxz@gmail.com
Iva Sella
ivanasea@gmail.com

¹ School of Information and Software Engineering,
University of Electronic Science and Technology of China,
Chengdu 610054, China

networks need to extend functionalities right to the edge, so that the analysis and processing of data from device to device can be handled. They also need to be fundamental in the environments they have been deployed to guarantee levels of interoperability and support for the IIoT devices. Cloud computing has provided an avenue where huge amounts of data can be handled (Truong and Dustdar 2015; Premkamal et al. 2018). This is because it has a robust presence always available even if one server failed. Such optimizations allow rich, heavy data to be delivered and accessed easily on the internet. Cloud computing, therefore, provides reliability on these networks, a performance that is good and also proves cost-effective.

In a nutshell, within the IIoT environment, the data is collected from sensors deployed in the manufacturing department and then uploaded through the internet to the cloud servers with the aim of making it easily accessible from anywhere (shown in Fig. 1). Unfortunately, the process of outsourcing data onto cloud servers results to a deprivation of the control users have over their data. Given an example where data already outsourced, could be changed, erased, modified or even completely damaged beyond recovery. To help avoid such scenarios, the integrity, confidentiality, and reliability of the data need to be ensured (Esposito et al. 2016). And a way to ensure such qualities are actualized, is to have data encrypted before it is outsourced to the cloud server.

Encrypting data can also ensure that the cloud service providers do not get to access or accidentally expose users' data. Users may later need to access their encrypted data and information and hence they will be prompted to search for it. A challenge arises from this scenario. The most ingenious option the users would have in order to retrieve their data would involve an initial download of all data, followed by decryption after which they can now search from it. This process would incur high costs when it comes to computation and communication, consume time, and would also be unrealistic when applied in a real-time application. To handle this challenge, Boneh et al. (2004) came up with the notion of incorporating searching using keywords with

the public key encryption for an effective way for users to retrieve their information effectively. Nevertheless, in the PKE-KS scheme, the cloud server can only compare encrypted keywords and trapdoors under the same public key. Considering that IIoT's nature is heterogeneous, the PKE-KS scheme is not suitable to search in the cloud server. To address this problem, Yang et al. (2010) considered the concept of public key encryption scheme with the inclusion of equality test (PKE-ET). In PKE-ET, the search functionality can be executed between two ciphertexts that have been encrypted with the use of the same public key as well as the use of different public keys.

Despite, the PKE-ET schemes fulfilling the search requirements in the cloud server, they are constructed under the public key infrastructure (PKI) system. Consequently, for every user, their public key should be authenticated by a certificate, specifically a digital signature which is generated by a trusted entity known as the certificate authority (CA). In view of limited resources when it comes to the sensor nodes in the IIoT, the overhead incurred by the certificate management undoubtedly degrade the efficiency of the IIoT. To handle this challenge, Ma (2016) came up with the concept of Identity-based encryption, which incorporated the aspect of equality test (IBE-ET). The IBE-ET scheme is constructed under a public key cryptosystem that was based on the identity of a user (referred to as ID-PKC). This scheme was able to eliminate the certificates found in the public key infrastructure (PKI) system by setting user's public key to their identity. Nonetheless, this scheme also experiences difficulties that come with key escrow (Hassan et al. 2018). This results from allocating decryption keys in the care of a third party. Meaning the third party entity will have access to a user's encrypted data and can access it anytime. To deal with the key escrow difficulty, Al-Riyami and Paterson (2003) came up with the idea of the public key cryptosystem being Certificateless (CL-PKC). In this cryptosystem, users possess private keys which are in two parts. The key generation center (KGC) produces the first part of the key, while the other part is produced by the user. It means that the KGC has partial access to a user's private key, ensuring the third parties do not have access to a user's data. By incorporating the idea of PKE-ET and CL-PKC, Qu et al. (2018) introduced the notion of CL-PKC with equality test (CL-PKC-ET). However, the CL-PKC-ET scheme supports one type of authorization to perform equivalence test between two ciphertexts.

1.1 Related works

Cloud computing is improving drastically and the development of information, storage modes and facilities are getting a lot of attention daily. Encryption therefore has become the most important aspect in guaranteeing data

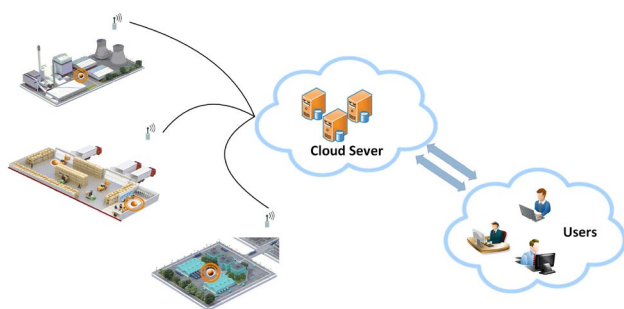


Fig. 1 A typical scenario for IIoT

security. With this, Boneh et al. (2004) preceded the concept of searching keywords into the Public Key Encryption (PKE-KS) scheme. In this scheme, the execution of equivalence test between keywords integrated in the various ciphertexts, by a semi-trusted third party was made possible. Also, Noroozi and Eslami (2019) proposed a generic construction for designing PEKS schemes which are secure against both online and offline keyword guessing attack (KGA) performed by outsiders. Subsequently, Baek et al. (2008) suggested PKE-KS that was efficient. The scheme was known as Identity-based encryption with keyword search (IBE-KS). The IBE-KS scheme managed to take away the secure channel that was incorporated within Boneh et al.'s scheme. Later on an extension of the PKE-KS was also proposed by Abdalla et al. (2005). In this scheme, the PKE-KS propagated to the Identity Based scope by adopting the pros of both IBE scheme and PKE scheme. IBE-KS supports only ciphertexts that have been encrypted by the use of the same public key. Unfortunately, when considered in the real-time application, the scheme is found to have limitations. The limitation of encryption being based on only the same public key was then solved by Yang et al. (2010). Yang proposed an encryption scheme that incorporated the equality test functionality. This functionality test was not only between two ciphertexts encrypted under the same public key but also under different public keys. It was referred to as Public Key Encryption with Equality Test (PKE-ET). PKE-ET has proved to be a very flexible search functionality by giving an authorized cloud server the permission to decide if, when given two ciphertexts, the equivalent messages that have been encrypted with either the same or different public keys are the same. With more and more researches aiming to better this scheme, Tang (2011) went ahead and suggested the fine-grained authorization Public key encryption with equality test (FG-PKE-ET). This scheme was intended to enforce authorization on the PKE-ET, by allowing only two users to perform the equality test but with assistance from a third entity. Tang still went ahead and introduced an improvement on the FG-PKE-ET with a two-proxy setting (Tang 2012a). Here two proxies work together to ensure that the equality test is performed. Furthermore, Tang went to propose a more refined scheme known as all-or-nothing PKE-ET (Tang 2012b). This scheme would have a mechanism to choose who can carry out the equality test but in a coarser granularity manner. Despite these schemes, the need to have only delegated parties having to complete the work in a practical multi-user setting arose. And this led to Ma et al. (2015b) introducing the notion of PKE incorporating delegated equality test (PKE-DET). Subsequently, Huang et al. (2015) proposed a scheme that involved authorized equality test i.e. PKE-AET. Here the given users have the

chance of testing equivalence between two ciphertexts or rather two specified ciphertexts. Ma et al. (2015a) went ahead and improved the PKE-AET by proposing a scheme that supported flexible authorization known as PKE-ET-FA. The urge to keep improving these schemes by researches resulted to introduction of the PKE-ET in to the 5G networks field by Xu et al. (2017). This scheme provides users with providence to check if a specified cloud server has correctly performed the equality test on the given ciphertexts. A closer investigation on the above mentioned schemes, we realize that all these schemes have their basis on the Public Key Infrastructure (PKI). The PKI is not very scalable, in that, the distribution of public keys is apparently unmanageable. Therefore to deal with the issue of managing these public key certificates, a new direction of research has been considered. (Ma 2016; Lee et al. 2016; Wu et al. 2017). This new direction is known as Identity-Based Encryption that has outsourced equality tests. This approach is aimed at providing flexible search functionality at the same time eradicate the issues that arise with managing certificates. However, these schemes suffer from the key escrow problem. To help control the key escrow issue, Qu et al. (2018) and Elhabob et al. (2019) introduced the notion of certificateless public key encryption with equality test (CL-PKE-ET). As of now we recognize, that the certificateless public key cryptography featured with authorized equivalence test has not been treated in literature so far.

1.2 Contributions

To enhance the privacy of the user and restrict the process of the equivalence test in the cloud server, an efficient public key cryptography scheme that is certificateless and incorporates authorized equality test is presented in this paper. In this suggested scheme, the authorization mechanism is versatile such that four types of authorizations (shown in Fig. 2) can be used to delegate servers in the cloud to execute the search functionality (see 2.1 for more detail). Our contribution can be briefly described as follows:

1. By integrating PKE-ET and CL-PKE, the certificateless public key encryption with authorized equality test (CL-PKE-AET) is presented. In CL-PKE-AET scheme, the authority is given to the cloud server to determine whether encryptions contain the equivalent messages. Moreover, the proposed scheme implements fine-grained access control to the cloud server.
2. The bilinear pairing has been considered for a concrete CL-PKC-AET scheme. Furthermore, we are basing our security on the BDH assumption and considering a random oracle model.

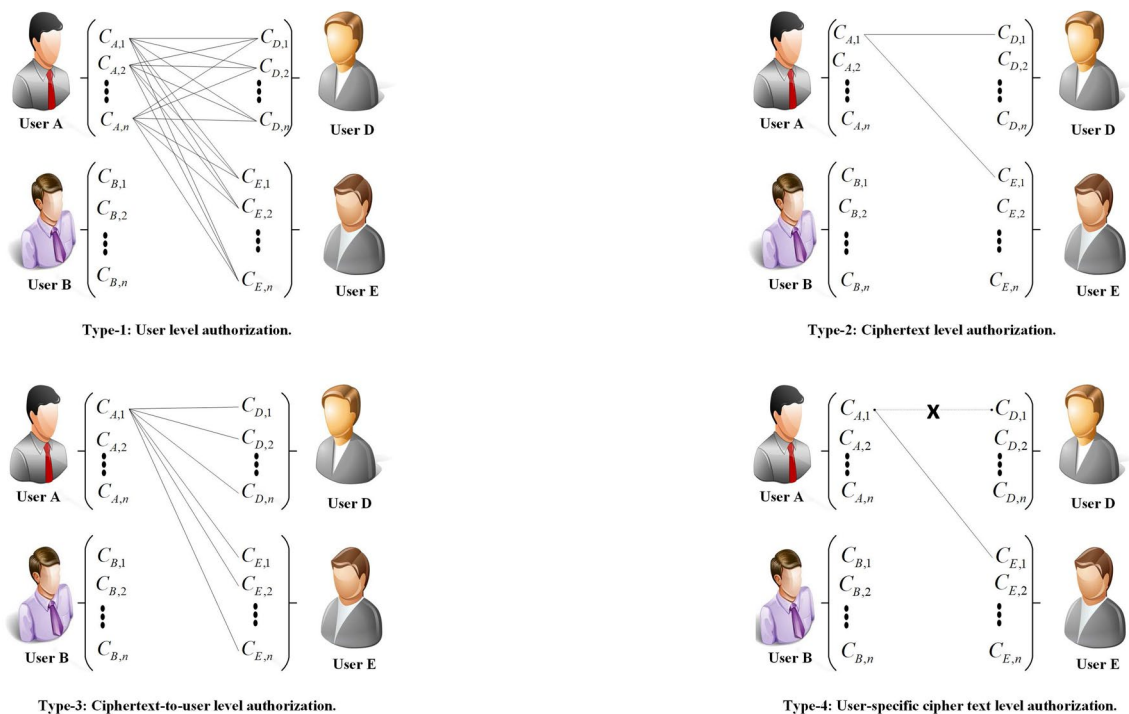


Fig. 2 Four types of authorizations for CL-PKC-AET

- Our scheme is compared with other alternatives. The result shows that the CL-PKC-AET scheme in this paper outperforms the existing works.

1.3 Organisation

The system model, as well as the adversary model, are presented in Sect. 2. In Sect. 3, the mathematical background and definitions of CL-PKC-AET scheme are explained. In Sect. 4, we focus on the concrete construction of CL-PKC-AET, while in Sect. 5 we focus on the security analysis of CL-PKC-AET. We go ahead to compare CL-PKC-AET with other alternatives in Sect. 6, and finally, we draw the paper to a close in Sect. 7.

2 System model and adversary model

In this section, the system model of the proposed scheme as well as the define of adversary model are presented.

2.1 System model

In this subsection, we present the proposed system model for the CL-PKC-AET scheme (shown Fig. 3). The system consists of four elements which are: key generation center (KGC), server in the cloud (CS), owners of data (DO), and users of the data (DU). The KGC creates the partial

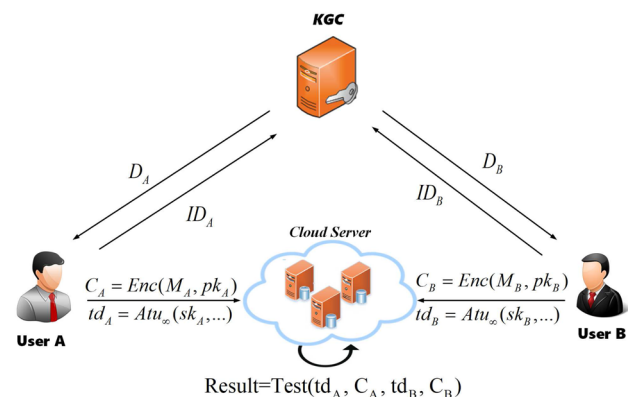


Fig. 3 System model for CL-PKC-AET scheme

secret key of the users according to the identity submitted to it by the user. Afterwards, the DO encrypts his sensing data and generates the trapdoor by using encryption and authorization algorithms, respectively. Then, the DO delivers the data that has been encrypted and the trapdoor to the CS. To search these encrypted data stored in CS, the DU sends his trapdoor to the CS. By using td_A and td_B the CS can check the equivalence between C_A and C_B . Finally, if $C_A = C_B$, the CS returns the searching result to the DU. Otherwise, CS returns 0.

In the proposed scheme, four kinds of authorizations are introduced to provide flexible equality test. Let user A,

user B, user D, and user E be the users of a system. Four kinds of authorizations are provided (shown in Fig. 2) as follows:

1. *Type-1* User level authorization: For this authorization, all the ciphertexts of user A can be compared with all the ciphertexts of user D and user E.
2. *Type-2* Ciphertext level authorization: For this authorization, we only compare a particular ciphertext of user A with a particular ciphertext of user E or user D.
3. *Type-3* Ciphertext-to-user level authorization: For this authorization, we can compare a specific ciphertext of user A with all the ciphertexts of user D and user E. This authorization is an incorporation of Type-1 and Type-2 authorizations.
4. *Type-4* User-specific cipher text level authorization: For this kind of authorization, we can compare a particular ciphertext of user A with a particular ciphertext of user E. Furthermore, the user A specifies which ciphertext to compare, and the user E can compare with him. In this way, user D can not make a comparison with a user A.

2.2 Adversary model

According to Qu et al. (2018), for the security of CL-PKC-AET four types of adversaries are considered. For simplicity, the Type-3 authorization is omitted, since Type-3 authorization is an incorporation of Type-1 authorization and Type-2 authorization. The four types of adversaries are described below:

1. *Type-1 adversary* This type of adversary $\mathcal{A}_1$ cannot acquire the master secret key. However, $\mathcal{A}_1$ can solicit and replace public keys with values it chooses by itself. Furthermore, given information of $Aut_{\mathbb{M}}(\mathbb{M} = 1, 2, 4)$, $\mathcal{A}_1$ should not be able to retrieve the plaintext from the challenge ciphertext. For this kind of adversary, one-way chosen ciphertext attack (OW-CCA) security for CL-PKC-AET is defined.
2. *Type-2 adversary* This type of adversary $\mathcal{A}_2$ can acquire the master secret key. However, $\mathcal{A}_2$ cannot replace public keys of any users. Furthermore, given information of $Aut_{\mathbb{M}}(\mathbb{M} = 1, 2, 4)$, $\mathcal{A}_2$ cannot retrieve the plaintext from the challenge ciphertext. With this type of adversary, OW-CCA security for CL-PKC-AET is defined.
3. *Type-3 adversary* This type of adversary $\mathcal{A}_3$ cannot get the master secret key. However, $\mathcal{A}_3$ may request and change public keys with values of its choosing. Furthermore, in the absence of given information of $Aut_{\mathbb{M}}(\mathbb{M} = 1, 2, 4)$, $\mathcal{A}_3$ cannot resolve whether the challenge ciphertext is the encryption of a given message. For this kind of adversary, indistinguishability chosen ciphertext attack (IND-CCA) security for CL-PKC-AET is considered.

4. *Type-4 adversary* This type of adversary $\mathcal{A}_4$ can acquire the master secret key. However, $\mathcal{A}_4$ cannot replace public keys of any users. Furthermore, in the absence of given information of $Aut_{\mathbb{M}}(\mathbb{M} = 1, 2, 4)$, $\mathcal{A}_4$ cannot resolve whether the challenge ciphertext is the encryption of a given message. For this kind of adversary, IND-CCA security for CL-PKC-AET is considered.

2.2.1 OW-CCA security for Type-1 adversary

The OW-CCA security for $Aut_{\mathbb{M}}(\mathbb{M} = 1, 2, 4)$ authorization against *Type-1 adversary* in CL-PKC-AET is defined below.

Game 1 Given that $\mathcal{A}_1$ is a *Type-1 adversary*. The game between $\mathcal{A}_1$ and the challenger is depicted as follows:

1. *Setup* The challenger runs the *Setup* algorithm to generate the public parameters $PubP$ and master secret key msk . Afterwards, the challenger gives $PubP$ to $\mathcal{A}_1$ and keeps msk secret. Furthermore, the challenger runs the **Extract-Partial-Secret-Key**, *Set-Secret-Value*, *Set-Secret-Key*, and *Set-Public-Key* algorithms to create n secret key sk_i associated with ID_i , where $(1 \leq i \leq n)$. Finally, the challenger gives all ID_i to $\mathcal{A}_1$.
2. *Phase 1* $\mathcal{A}_1$ is allowed to issue the following queries for polynomially many times.
 - *Partial secret key queries* $\langle ID_i \rangle$ The challenger sends D_i to $\mathcal{A}_1$.
 - *Secret key queries* $\langle ID_i \rangle$ The challenger sends sk_i to $\mathcal{A}_1$.
 - *Public key queries* $\langle ID_i \rangle$ The challenger sends pk_i to $\mathcal{A}_1$.
 - *Replace public key queries* $\langle ID_i, pk'_i \rangle$: the challenger replaces the public key pk of the corresponding user with pk'_i .
 - *Decryption queries* $\langle ID_i, C_i \rangle$ The challenger executes the algorithm $Decrypt(C_i, sk_i)$, where sk_i is the secret key associated with ID_i . Eventually, the challenger gives M_i to $\mathcal{A}_1$.
 - *Authorization queries for* $Aut_{\mathbb{M}}(\mathbb{M} = 1, 2, 4)$
 - *Type-1 Authorization* $\langle ID_i \rangle$: The challenger responds with $td_{i,1}$.
 - *Type-2 Authorization* $\langle ID_i, C_i \rangle$ The challenger responds with $td_{i,2}$.
 - *Type-4 Authorization* $\langle ID_i, C_i, ID_j, C_j \rangle$ The challenger responds with $td_{i,4}$.
3. *Challenge* The challenger randomly picks the plaintext $M \in \mathbb{G}_1^*$ and computes $C^* = Encrypt(ID_{ch}, M)$. Eventually, the challenger gives C^* to $\mathcal{A}_1$ as its challenge ciphertext.

4. *Phase 2* The challenger responds to $\mathcal{A}_1$ in the same way as in *Phase 1* on the grounds that:
 - ID_{ch} is not queried in the *Secret key queries*.
 - If the public key associated with ID_{ch} is replaced, the ID_{ch} should not be queried in the *Partial secret key queries*.
 - If the public key of the user is replaced, the corresponding identity ID_i should not be queried in the *Secret key queries*.
 - (ID_{ch}, C^*) is not queried in the *Decryption queries*.

5. *Guess* $\mathcal{A}_1$ outputs M' , and wins if $M' = M$. The advantage of $\mathcal{A}_1$ in the game above is defined as follows:

$$Adv_{CL-PKC-AET, \mathcal{A}_1}^{OW-CCA, Type_{\mathbb{M}}}(\lambda) = Pr[M' = M](\mathbb{M} = 1, 2, 4).$$

Definition 1 The CL-PKC-AET scheme is OW-CCA secure for $Type_{\mathbb{M}}(\mathbb{M} = 1, 2, 4)$ authorization if for all OW-CCA adversary $\mathcal{A}_1$, its advantage

$Adv_{CL-PKC-AET, \mathcal{A}_1}^{OW-CCA, Type_{\mathbb{M}}}(\lambda)$ is negligible in the security parameter λ .

2.2.2 OW-CCA security for Type-2 adversary

The OW-CCA security for $Aut_{\mathbb{M}}(\mathbb{M} = 1, 2, 4)$ authorization against *Type-2 adversary* in CL-PKC-AET is shown as follows:

Game 2 Given that $\mathcal{A}_2$ is a *Type-2 adversary*. The game between $\mathcal{A}_2$ and the challenger is depicted as follows:

1. *Setup* The challenger runs the *Setup* algorithm to generate the public parameters $PubP$ and master secret key msk . Afterwards, the challenger sends $PubP$ and the msk to $\mathcal{A}_2$. Furthermore, the challenger runs the *Extract-Partial-Secret-Key*, *Set-Secret-Value*, *Set-Secret-Key*, and *Set-Public-Key* algorithms to create n secret key sk_i associated with ID_i , where $(1 \leq i \leq n)$. Finally, the challenger gives all ID_i to $\mathcal{A}_2$.
2. *Phase 1* $\mathcal{A}_2$ issues queries as in *Game 1*, except the *Partial secret key queries* and the *Replace public key queries* are not allowed to issue in this game.
3. *Challenge* The challenger randomly picks the plaintext $M \in \mathbb{G}_1^*$ and computes $C^* = \text{Encrypt}(ID_{ch}, M)$. Eventually, the challenger gives C^* to $\mathcal{A}_2$ as its challenge ciphertext.
4. *Phase 2* The challenger responds to $\mathcal{A}_2$ in the same way as in *Phase 1* on grounds that:
 - ID_{ch} is not queried in the *Secret key queries*.
 - (ID_{ch}, C^*) is not queried in the *Decryption queries*.

5. *Guess* $\mathcal{A}_2$ outputs M' , and wins if $M' = M$. therefore $\mathcal{A}_2$ advantage in the game is:

$$Adv_{CL-PKC-AET, \mathcal{A}_2}^{OW-CCA, Type_{\mathbb{M}}}(\lambda) = Pr[M' = M](\mathbb{M} = 1, 2, 4).$$

Definition 2 The CL-PKC-AET scheme is OW-CCA secure for $Type_{\mathbb{M}}(\mathbb{M} = 1, 2, 4)$ authorization for all OW-CCA adversary $\mathcal{A}_2$, if its advantage

$Adv_{CL-PKC-AET, \mathcal{A}_2}^{OW-CCA, Type_{\mathbb{M}}}(\lambda)$ is negligible in the security parameter λ .

2.2.3 IND-CCA security for Type-3 adversary

The IND-CCA security for $Aut_{\mathbb{M}}(\mathbb{M} = 1, 2, 4)$ authorization against *Type-3 adversary* in CL-PKC-AET is shown as follows:

Game 3 Given that $\mathcal{A}_3$ is a *Type-3 adversary*. The game between $\mathcal{A}_3$ and the challenger is depicted as follows:

1. *Setup* The challenger runs the *Setup* algorithm to generate the public parameters $PubP$ and master secret key msk . Afterwards, the challenger gives $PubP$ to $\mathcal{A}_3$ and keeps msk secret. Furthermore, the challenger runs the *Extract-Partial-Secret-Key*, *Set-Secret-Value*, *Set-Secret-Key*, and *Set-Public-Key* algorithms to create n secret key sk_i associated with ID_i , where $(1 \leq i \leq n)$. Finally, the challenger gives all ID_i to $\mathcal{A}_3$.
2. *Phase 1* $\mathcal{A}_3$ issues queries as in *Game 1*.
3. *Challenge* $\mathcal{A}_3$ gives the challenger two plaintexts of equal-length $M_0, M_1 \in \mathbb{G}_1^*$. The challenger then selects a random bit $b \in \{0, 1\}$ and computes $C^* = \text{Encrypt}(M_b, ID_{ch})$. The challenger then sends C^* to $\mathcal{A}_3$ as the challenge ciphertext.
4. *Phase 2* The challenger responds to $\mathcal{A}_3$ in a similar way as in *Phase 1* on grounds that:
 - ID_{ch} is not queried in the *Secret key queries*.
 - If the public key associated with ID_{ch} is replaced, the ID_{ch} is not queried in the *Partial secret key queries*.
 - If the public key of the user is replaced, the corresponding identity ID_i should not be queried in the *Secret key queries*.
 - (ID_{ch}, C^*) should not be queried in the *Decryption queries*.
 - ID_{ch} should not be queried in the *Authorization queries*.
5. *Guess* $\mathcal{A}_3$ outputs $b' \in \{0, 1\}$, and wins if $b' = b$. The advantage of $\mathcal{A}_3$ in the game above is depicted as follows:

$$\text{Adv}_{\text{CL-PKC-AET}, \mathcal{A}_3}^{\text{IND-CCA}, \text{Type}_{\mathcal{A}_3}}(\lambda) = \left| \Pr[b' = b] - \frac{1}{2} \right| (\mathcal{A} = 1, 2, 4).$$

Definition 3 The CL-PKC-AET scheme is IND-CCA secure for $\text{Type}_{\mathcal{A}_3}(\mathcal{A} = 1, 2, 4)$ authorization if for all IND-CCA adversary $\mathcal{A}_3$, its advantage

$\text{Adv}_{\text{CL-PKC-AET}, \mathcal{A}_3}^{\text{OW-CCA}, \text{Type}_{\mathcal{A}_3}}(\lambda)$ is negligible in the security parameter λ .

2.2.4 IND-CCA security for Type-4 adversary

The IND-CCA security for $\text{Aut}_{\mathcal{A}_4}(\mathcal{A} = 1, 2, 4)$ authorization against *Type-4 adversary* in CL-PKC-AET is defined below.

Game 4 Suppose that $\mathcal{A}_4$ be a *Type-4 adversary*. The game between $\mathcal{A}_4$ and the challenger is depicted as follows:

1. *Setup* The challenger runs the *Setup* algorithm to generate the public parameters PubP and master secret key msk . Afterwards, challenger sends PubP and msk to $\mathcal{A}_4$. Furthermore, the challenger runs the *Extract-Partial-Secret-Key*, *Set-Secret-Value*, *Set-Secret-Key*, and *Set-Public-Key* algorithms to create n secret key sk_i associated with ID_i , where $(1 \leq i \leq n)$. Finally, the challenger gives all ID_i to $\mathcal{A}_4$.
2. *Phase 1* $\mathcal{A}_4$ issues queries as in *Game 1*, except the *Partial secret key queries* and the *Replace public key queries* are not allowed to issue in this game.
3. *Challenge* $\mathcal{A}_4$ gives the challenger two plaintexts of equal-length $M_0, M_1 \in \mathbb{G}_1^*$. The challenger selects a random bit $b \in \{0, 1\}$ and computes $C^* = \text{Encrypt}(M_b, ID_{ch})$. After that, the challenger sends C^* to $\mathcal{A}_3$ as the challenge ciphertext.
4. *Phase 2* The challenger responds to $\mathcal{A}_4$ in a similar way as in *Phase 1* on grounds that:
 - ID_{ch} is not queried in the *Secret key queries*.
 - (ID_{ch}, C^*) is not queried in the *Decryption queries*.
 - ID_{ch} is not queried in the *Authorization queries*.
5. *Guess* $\mathcal{A}_4$ outputs $b' \in \{0, 1\}$, and wins if $b' = b$. The advantage of $\mathcal{A}_4$ in the game above is depicted as follows:

$$\text{Adv}_{\text{CL-PKC-AET}, \mathcal{A}_4}^{\text{IND-CCA}, \text{Type}_{\mathcal{A}_4}}(\lambda) = \left| \Pr[b' = b] - \frac{1}{2} \right| (\mathcal{A} = 1, 2, 4).$$

Definition 4 The CL-PKC-AET scheme is IND-CCA secure for $\text{Type}_{\mathcal{A}_4}(\mathcal{A} = 1, 2, 4)$ authorization if for all IND-CCA adversary $\mathcal{A}_4$, its advantage

$\text{Adv}_{\text{CL-PKC-AET}, \mathcal{A}_4}^{\text{OW-CCA}, \text{Type}_{\mathcal{A}_4}}(\lambda)$ is negligible in the security parameter λ .

3 Preliminaries

In this section, the mathematical background related to the proposed scheme as well as the definition of CL-PKC-AET are presented.

3.1 Mathematical background

3.1.1 Bilinear pairing

Given $\mathbb{G}_1, \mathbb{G}_2$, are two multiplicative cyclic groups both having the same prime order p . g is a generator of $\mathbb{G}_1$. A bilinear pairing $e : \mathbb{G}_1 \times \mathbb{G}_1 \rightarrow \mathbb{G}_2$ is required as follows:

1. *Bilinearity* $\forall g, h \in \mathbb{G}_1$ and $\forall a, b \in \mathbb{Z}_p^*$, $e(g^a, h^b) = e(g, h)^{ab}$.
2. *Non-degeneracy* $\forall g, h \in \mathbb{G}_1$, $e(g, h) \neq 1$.
3. *Computability* There is an efficient algorithm to calculate $e(g, h)$, $\forall g, h \in \mathbb{G}_1$.

3.1.2 Bilinear Diffie-Hellman (BDH) problem

Randomly choose $a, b, c \in \mathbb{Z}_p^*$, the BDH problem holds that for any polynomial-time algorithm $\mathcal{A}$ which wants to find $e(g, g)^{abc}$ from $\langle g, g^a, g^b, g^c \rangle$, the advantage is negligible.

3.2 Definition of CL-PKC-AET

A certificateless public key cryptography scheme that entails authorized equality test is defined by the following algorithms:

1. *Setup* The *PGK* runs this algorithm. It uses λ as input for the security parameter and outputs the system parameters PubP and a master secret key msk .
2. *Extract-Partial-Secret-Key* The *PGK* also runs this algorithm. It uses as input PubP , msk , and a public identity of a user $ID \in \{0, 1\}^*$. It returns a partial secret key D_{ID} .
3. *Set-Secret-Value* The user runs this algorithm. It uses as inputs PubP and ID , and returns user's secret value x_{ID} .
4. *Set-Secret-Key* The user runs this algorithm. It uses as inputs PubP , D_{ID} , and x_{ID} . It returns user's secret key sk .
5. *Set-Public-Key* The user runs this algorithm. It takes as inputs PubP and sk . It returns user's public key pk .
6. *Encrypt* This algorithm uses as inputs PubP , a message M , and public key pk . It returns a ciphertext C .
7. *Decrypt* This algorithm uses as inputs PubP , a ciphertext C , and secret key sk . It returns the plaintext M . Otherwise, it returns the symbol $\perp$. Let A and B be two users of the system.

We denote by $C_A = (C_{A,1}, C_{A,2}, C_{A,3}, C_{A,4})$ and $C_B = (C_{B,1}, C_{B,2}, C_{B,3}, C_{B,4})$ the ciphertexts of A and B, respectively. The $Aut_{\bowtie}(\bowtie = 1, 2, 3, 4)$ algorithm is defined to produce a trapdoor for A's ciphertext(s) required to be compared with B's ciphertext(s), and $Test_{\bowtie}(\bowtie = 1, 2, 3, 4)$ algorithm to ascertain whether ciphertexts belonging to two users contain the same message. The four types of authorizations are defined as follows.

8. *Type-1: Authorization*

- $Aut_1(sk_A)$ The algorithm uses A's secret key sk_A as input and outputs a trapdoor $td_{A,1}$ for A.
- $Test_1(C_A, td_{A,1}, C_B, td_{B,1})$ This algorithm takes as inputs A's ciphertexts C_A , trapdoor $td_{A,1}$ and B's ciphertexts C_B , trapdoor $td_{B,1}$. The $Test_1$ algorithm outputs 1 if C_A and C_B contain the same message. Otherwise, it returns 0.

9. *Type-2: Authorization*

- $Aut_2(sk_A, C_A)$: The algorithm uses a secret key sk_A and the ciphertext C_A of user A as input. The Aut_2 algorithm outputs a trapdoor $td_{A,2}$ for (A, C_A) .
- $Test_2(C_A, td_{A,2}, C_B, td_{B,2})$: This algorithm uses as inputs A's ciphertext C_A , trapdoor $td_{A,2}$ and B's ciphertext C_B , trapdoor $td_{B,2}$. The $Test_2$ algorithm outputs 1 if C_A and C_B possess the same message. Otherwise, it returns 0.

10. *Type-3: Authorization*

- $Aut_{A,3}(sk_A, C_A)$ The algorithm uses a secret key sk_A and the ciphertext C_A of user A as input. The $Aut_{A,3}$ algorithm outputs a trapdoor $td_{A,3}$ for (A, C_A) .
- $Aut_{B,3}(sk_B)$ This algorithm uses as an input B's secret key sk_B and outputs a trapdoor $td_{B,3}$ for user B.
- $Test_3(C_A, td_{A,3}, C_B, td_{B,3})$: This algorithm uses as inputs A's ciphertext C_A , trapdoor $td_{A,3}$ and B's ciphertexts C_B , trapdoor $td_{B,3}$. The $Test_3$ algorithm outputs 1 if C_A and C_B contain the same message. Otherwise, it returns 0.

11. *Type-4: Authorization*

- $Aut_4(sk_A, C_A, C_B)$ The algorithm uses a secret key sk_A of user A, a ciphertext C_A of user A, and a ciphertext C_B of user B as inputs. The Aut_4 algorithm outputs a trapdoor $td_{A,4} = (T_{A,1}, T_{A,2})$ for (A, C_A, C_B) .
- $Test_4(C_A, td_{A,4}, C_B, td_{B,4})$ This algorithm uses as inputs A's ciphertext C_A , trapdoor $td_{A,4}$ and B's ciphertexts C_B , trapdoor $td_{B,4}$. The $Test_4$ algorithm

outputs 1 if C_A and C_B contain the same message. Otherwise, it returns 0.

4 Concrete construction

In this section, the concrete construction of CL-PKC-AET scheme is presented by the following algorithms.

1. *Setup* Given a secure parameter λ , the *Setup* algorithm proceeds as follows:

- It generates the pairing parameters: two groups $\mathbb{G}_1, \mathbb{G}_2$ of prime order p , and an admissible bilinear map $e : \mathbb{G}_1 \times \mathbb{G}_1 \rightarrow \mathbb{G}_2$. Then choose a random generator $g \in \mathbb{G}_1$.
- It then chooses cryptographic hash functions: $H_1 : \{0, 1\}^* \rightarrow \mathbb{G}_1, H_2 : \mathbb{G}_2 \rightarrow \mathbb{G}_1, H_3 : \mathbb{G}_2 \rightarrow \{0, 1\}^{n_1+n_2}$, where $n_1 = |\mathbb{G}_1|$ and $n_2 = |\mathbb{Z}_p^*|$.
- Randomly choose $(s_1, s_2) \in \mathbb{Z}_p^*$, then set $g_1 = g^{s_1}$ and $g_2 = g^{s_2}$. The public parameters are $\langle p, \mathbb{G}_1, \mathbb{G}_2, e, g, g_1, g_2, H_1, H_2, H_3 \rangle$.

2. *Extract-Partial-Secret-Key* Given a string $ID \in \{0, 1\}^*$:

- It computes $h_{ID} = H_1(ID) \in \mathbb{G}_1$.
- It then computes partial secret key $D_{ID} = (D_1, D_2) = (h_{ID}^{s_1}, h_{ID}^{s_2})$, where (s_1, s_2) is the master secret key.

3. *Set-Secret-Value* The algorithm uses as inputs $PubP$ and D_{ID} . It chooses $x_{ID} \in \mathbb{Z}_p^*$ randomly then returns x_{ID} as a secret value.

4. *Set-Secret-Key* The algorithm uses as inputs $PubP, D_{ID}$, and x_{ID} . It computes $sk = (sk_1, sk_2) = (D_1^{x_{ID}}, D_2^{x_{ID}})$

5. *Set-Public-Key* This algorithm takes as inputs $PubP$ and a secret value x_{ID} . It returns public key $pk = (X, pk_1, pk_2) = (g^{x_{ID}}, g_1^{x_{ID}}, g_2^{x_{ID}})$.

6. *Encrypt*: This algorithm proceeds as follows:

- Take the message $M \in \mathbb{G}_1^*$, the identity ID , and the public key $pk = (X, pk_1, pk_2)$ as inputs.
- Verify that if $pk = (X, pk_1, pk_2) \in \mathbb{G}_1^*$ and $e(X, g_1) = e(pk_1, g)$ and $e(X, g_2) = e(pk_2, g)$. If these verifications pass, do the encryption. Otherwise, terminate the encryption.
- Compute $h_{ID} = H_1(ID) \in \mathbb{G}_1^*$.
- Pick two random numbers $(r_1, r_2) \in \mathbb{Z}_p^*$.

- Compute $C = (C_1, C_2, C_3, C_4)$, where $C_1 = g^{r_1}$, $C_2 = g^{r_2}$, $C_3 = M^{r_1} \cdot H_2(e(h_{ID}, pk_1)^{r_1})$, and $C_4 = (M || r_1) \oplus H_3(e(h_{ID}, pk_2)^{r_2})$.
- $e(h_{ID}, pk_1)^{r_1}$ and $e(h_{ID}, pk_2)^{r_2}$ can be pre-calculated.

7. *Decrypt* This algorithm uses as inputs a ciphertext C and a secret key sk . It returns the plaintext M by working as follows.

- Compute $C_4 \oplus H_3(e(sk_2, C_2)) = C_4 \oplus H_3(e(D_2^{x_{ID}}, g^{r_2})) = C_4 \oplus H_3(e(h_{ID}, g_2^{x_{ID}})^{r_2}) = (M || r_1)$.
- Verify if $C_1 = g^{r_1}$ and $\frac{C_3}{M^{r_1}} = H_2(e(sk_1, C_1))$.
- If both verifications pass, return M . Otherwise, return the symbol $\perp$.

8. *Type-1: Authorization*

- $Aut_1(sk_A) = sk_{A,1} = td_{A,1}$.
- $Test_1(C_A, td_{A,1}, C_B, td_{B,1})$:

$$\begin{aligned} X_A &= \frac{C_{A,3}}{H_2(e(td_{A,1}, C_{A,1}))} \\ &= \frac{M_A^{r_{A,1}} \cdot H_2(e(h_{ID,A}, pk_{A,1})^{r_{A,1}})}{H_2(e(sk_{A,1}, C_{A,1}))} \\ &= \frac{M_A^{r_{A,1}} \cdot H_2(e(h_{ID,A}, g_1^{x_{ID,A}})^{r_{A,1}})}{H_2(e(D_{A,1}^{x_{ID,A}}, g^{r_{A,1}}))} \\ &= \frac{M_A^{r_{A,1}} \cdot H_2(e(h_{ID,A}, g_1^{x_{ID,A}})^{r_{A,1}})}{H_2(e(h_{ID,A}, g_1^{x_{ID,A}})^{r_{A,1}})} \\ &= M_A^{r_{A,1}} \end{aligned}$$

$$\begin{aligned} X_B &= \frac{C_{B,3}}{H_2(e(td_{B,1}, C_{B,1}))} \\ &= \frac{M_B^{r_{B,1}} \cdot H_2(e(h_{ID,B}, pk_{B,1})^{r_{B,1}})}{H_2(e(sk_{B,1}, C_{B,1}))} \\ &= \frac{M_B^{r_{B,1}} \cdot H_2(e(h_{ID,B}, g_1^{x_{ID,B}})^{r_{B,1}})}{H_2(e(D_{B,1}^{x_{ID,B}}, g^{r_{B,1}}))} \\ &= \frac{M_B^{r_{B,1}} \cdot H_2(e(h_{ID,B}, g_1^{x_{ID,B}})^{r_{B,1}})}{H_2(e(h_{ID,B}, g_1^{x_{ID,B}})^{r_{B,1}})} \\ &= M_B^{r_{B,1}} \end{aligned}$$

$Test_1$ algorithm returns 1 if $e(X_A, C_{B,1}) = e(X_B, C_{A,1})$. Otherwise, it returns 0.

9. *Type-2: Authorization*

- $Aut_2(sk_A, C_A) = H_2(e(sk_{A,1}, C_{A,1})) = td_{A,2}$.
- $Test_2(C_A, td_{A,2}, C_B, td_{B,2})$:

$$\begin{aligned} X_A &= \frac{C_{A,3}}{td_{A,2}} \\ &= \frac{M_A^{r_{A,1}} \cdot H_2(e(h_{ID,A}, pk_{A,1})^{r_{A,1}})}{H_2(e(sk_{A,1}, C_{A,1}))} \\ &= \frac{M_A^{r_{A,1}} \cdot H_2(e(h_{ID,A}, g_1^{x_{ID,A}})^{r_{A,1}})}{H_2(e(D_{A,1}^{x_{ID,A}}, g^{r_{A,1}}))} \\ &= \frac{M_A^{r_{A,1}} \cdot H_2(e(h_{ID,A}, g_1^{x_{ID,A}})^{r_{A,1}})}{H_2(e(h_{ID,A}, g_1^{x_{ID,A}})^{r_{A,1}})} \\ &= M_A^{r_{A,1}} \\ X_B &= \frac{C_{B,3}}{td_{B,2}} \\ &= \frac{M_B^{r_{B,1}} \cdot H_2(e(h_{ID,B}, pk_{B,1})^{r_{B,1}})}{H_2(e(sk_{B,1}, C_{B,1}))} \\ &= \frac{M_B^{r_{B,1}} \cdot H_2(e(h_{ID,B}, g_1^{x_{ID,B}})^{r_{B,1}})}{H_2(e(D_{B,1}^{x_{ID,B}}, g^{r_{B,1}}))} \\ &= \frac{M_B^{r_{B,1}} \cdot H_2(e(h_{ID,B}, g_1^{x_{ID,B}})^{r_{B,1}})}{H_2(e(h_{ID,B}, g_1^{x_{ID,B}})^{r_{B,1}})} \\ &= M_B^{r_{B,1}} \end{aligned}$$

$Test_2$ algorithm returns 1 if $e(X_A, C_{B,1}) = e(X_B, C_{A,1})$. Otherwise, it returns 0.

10. *Type-3: Authorization*

- $Aut_{A,3}(sk_A, C_A) = H_2(e(sk_{A,1}, C_{A,1})) = td_{A,3}$.
- $Aut_{B,3}(sk_B, C_B) = td_{B,3}$.
- $Test_3(C_A, td_{A,3}, C_B, td_{B,3})$:

$$\begin{aligned}
X_A &= \frac{C_{A,3}}{td_{A,3}} \\
&= \frac{M_A^{r_{A,1}} \cdot H_2(e(h_{ID,A}, pk_{A,1})^{r_{A,1}})}{H_2(e(sk_{A,1}, C_{A,1}))} \\
&= \frac{M_A^{r_{A,1}} \cdot H_2(e(h_{ID,A}, g_1^{x_{ID,A}})^{r_{A,1}})}{H_2(e(D_{A,1}^{x_{ID,A}}, g^{r_{A,1}}))} \\
&= \frac{M_A^{r_{A,1}} \cdot H_2(e(h_{ID,A}, g_1^{x_{ID,A}})^{r_{A,1}})}{H_2(e(h_{ID,A}, g_1^{x_{ID,A}})^{r_{A,1}})} \\
&= M_A^{r_{A,1}} \\
X_B &= \frac{C_{B,3}}{H_2(e(td_{B,3}, C_{B,1}))} \\
&= \frac{M_B^{r_{B,1}} \cdot H_2(e(h_{ID,B}, pk_{B,1})^{r_{B,1}})}{H_2(e(sk_{B,1}, C_{B,1}))} \\
&= \frac{M_B^{r_{B,1}} \cdot H_2(e(h_{ID,B}, g_1^{x_{ID,B}})^{r_{B,1}})}{H_2(e(D_{B,1}^{x_{ID,B}}, g^{r_{B,1}}))} \\
&= \frac{M_B^{r_{B,1}} \cdot H_2(e(h_{ID,B}, g_1^{x_{ID,B}})^{r_{B,1}})}{H_2(e(h_{ID,B}, g_1^{x_{ID,B}})^{r_{B,1}})} \\
&= M_B^{r_{B,1}}
\end{aligned}$$

$Test_3$ algorithm returns 1 if $e(X_A, C_{B,1}) = e(X_B, C_{A,1})$. Otherwise, it returns 0.

11. *Type-4: Authorization* In this kind of authorization, the user A chooses secret number z_A from $\mathbb{Z}_p^*$ randomly. In addition, the user B chooses secret number z_B from $\mathbb{Z}_p^*$ randomly. Afterwards, A and B are generate their trapdoors as follows:

- $Aut_4(sk_A, C_A, C_B, z_A) = td_{A,4} = (T_{A,1}, T_{A,2})$
 $= (\frac{g^{z_A}}{H_2(e(sk_{A,1}, C_{A,1}))}, e(g^{z_A}, C_{B,1})).$
- $Test_4(C_A, td_{A,4}, C_B, td_{B,4}).$

$$\begin{aligned}
X_A &= C_{A,3} \cdot T_{A,1} \\
&= \frac{M_A^{r_{A,1}} \cdot H_2(e(h_{ID,A}, pk_{A,1})^{r_{A,1}}) \cdot g^{z_A}}{H_2(e(sk_{A,1}, C_{A,1}))} \\
&= \frac{M_A^{r_{A,1}} \cdot H_2(e(h_{ID,A}, g_1^{x_{ID,A}})^{r_{A,1}}) \cdot g^{z_A}}{H_2(e(D_{A,1}^{x_{ID,A}}, g^{r_{A,1}}))} \\
&= \frac{M_A^{r_{A,1}} \cdot H_2(e(h_{ID,A}, g_1^{x_{ID,A}})^{r_{A,1}}) \cdot g^{z_A}}{H_2(e(h_{ID,A}, g_1^{x_{ID,A}})^{r_{A,1}})} \\
&= M_A^{r_{A,1}} \cdot g^{z_A}. \\
X_B &= C_{B,3} \cdot T_{B,1} \\
&= \frac{M_B^{r_{B,1}} \cdot H_2(e(h_{ID,B}, pk_{B,1})^{r_{B,1}}) \cdot g^{z_B}}{H_2(e(sk_{B,1}, C_{B,1}))} \\
&= \frac{M_B^{r_{B,1}} \cdot H_2(e(h_{ID,B}, g_1^{x_{ID,B}})^{r_{B,1}}) \cdot g^{z_B}}{H_2(e(D_{B,1}^{x_{ID,B}}, g^{r_{B,1}}))} \\
&= \frac{M_B^{r_{B,1}} \cdot H_2(e(h_{ID,B}, g_1^{x_{ID,B}})^{r_{B,1}}) \cdot g^{z_B}}{H_2(e(h_{ID,B}, g_1^{x_{ID,B}})^{r_{B,1}})} \\
&= M_B^{r_{B,1}} \cdot g^{z_B}.
\end{aligned}$$

Correctness

$$\begin{aligned}
&\frac{e(X_A, C_{B,1})}{e(X_B, C_{A,1})} \\
&= \frac{e(M_A^{r_{A,1}} \cdot g^{z_A}, C_{B,1})}{e(M_B^{r_{B,1}} \cdot g^{z_B}, C_{A,1})} \\
&= \frac{e(M_A^{r_{A,1}}, C_{B,1}) \cdot e(g^{z_A}, C_{B,1})}{e(M_B^{r_{B,1}}, C_{A,1}) \cdot e(g^{z_B}, C_{A,1})} \\
&= \frac{e(M_A, g)^{r_{A,1} r_{B,1}} \cdot e(g^{z_A}, C_{B,1})}{e(M_B, g)^{r_{A,1} r_{B,1}} \cdot e(g^{z_B}, C_{A,1})}.
\end{aligned}$$

Thus, $\frac{e(X_A, C_{B,1})}{e(X_B, C_{A,1})} = \frac{T_{A,2}}{T_{B,2}} = \frac{e(g^{z_A}, C_{B,1})}{e(g^{z_B}, C_{A,1})}$ holds and the $Test_4$ algorithm returns 1 if $M_A = M_B$. Otherwise, it returns 0.

5 Security analysis

Theorem 5.1 Suppose that H_1, H_2, H_3 are random oracles and assume that the BDH problem is hard. Therefore, our CL-PKC-AET is OW-CCA secure. Significantly, assume there is a *Type-1 adversary* $\mathcal{A}_1$ that has advantages $\epsilon_1(\lambda)$ against the CL-PKC-AET. Assume that $\mathcal{A}_1$ makes q_{pk} public key query, q_{sk} secret key query, q_{psk} partial secret key query, q_{aut} authorization query for $type_{\bowtie} (\bowtie = 1, 2, 4)$, q_{rp} replace public key query, q_{dec} decryption query, q_{H_2} hash query to H_2 , and q_{H_3} hash query to H_3 . Thus, we have an algorithm $\mathcal{B}_1$ which breaks BDH problem with advantage at least $\epsilon_1(\lambda)/e(q_{sk} + q_{psk} + q_{aut} + 1) \cdot q_{H_3}$.

Proof Suppose that $\mathcal{B}_1$ is given as inputs the BDH parameters $\langle p, \mathbb{G}_1, \mathbb{G}_2, e \rangle$ and arbitrary instance $\langle g, g^a, g^b, g^c \rangle$ of the BDH problem, where g is a random generator of $\mathbb{G}_1$ and $a, b, c \in_R \mathbb{Z}_p^*$. Suppose that $e(g, g)^{abc} \in \mathbb{G}_2$ is the solution of BDH problem. Then, we demonstrate how an algorithm $\mathcal{B}_1$ obtains $e(g, g)^{abc}$ by reacting with $\mathcal{A}_1$ as follows:

1. *Setup* The algorithm $\mathcal{B}_1$ gives $\mathcal{A}_1$ the public parameters $PubP = \langle p, \mathbb{G}_1, \mathbb{G}_2, e, g, g_1, g_2, H_1, H_2, H_3 \rangle$, where $g_1 = g_2 = g^a$.
2. *Phase 1* At any time $\mathcal{A}_1$ can make query to $q_{pk}, q_{sk}, q_{psk}, q_{aut}, q_{rp}, q_{dec}, q_{H_2}$, or q_{H_3} . To respond these queries $\mathcal{B}_1$ works as follows:
 - *H_1 -query(ID_i)* $\mathcal{B}_1$ creates a list of tuples $\langle ID_i, d_i, z_i, c_i \rangle$ denoted by H_1^{list} . Here, $c_i \in \{0, 1\}$, where 1 represents the probability of δ and 0 represents the probability of $1 - \delta$. If $c_i = 0$, $\mathcal{B}_1$ returns $d_i = (g_1^{z_i}, g_2^{z_i})$. Otherwise, if $c_i = 1$, $\mathcal{B}_1$ returns $d_i = (g_1^{b_{z_i}}, g_2^{b_{z_i}})$.
 - *H_2 -query(w_i)* $\mathcal{B}_1$ creates a list of tuples $\langle w_i, l_i \rangle$ denoted by H_2^{list} . If w_i is already stored in H_2^{list} , $\mathcal{B}_1$ responds with $H_2(w_i) = l_i$. Otherwise, $\mathcal{B}_1$ chooses $l_i \in_R \mathbb{G}_1$ and records item $[w_i, l_i]$ in H_2^{list} . Then, $\mathcal{B}_1$ returns l_i to $\mathcal{A}_1$.
 - *H_3 -query(v_i)* $\mathcal{B}_1$ creates a list of tuples $\langle v_i, h_i \rangle$ denoted by H_3^{list} . If v_i is already stored in H_3^{list} , $\mathcal{B}_1$ responds with $H_3(v_i) = h_i$. Otherwise, $\mathcal{B}_1$ chooses $h_i \in_R \{0, 1\}^{n_1+n_2}$ and records item $[v_i, h_i]$ in H_3^{list} . Then, $\mathcal{B}_1$ returns h_i to $\mathcal{A}_1$.
 - *Public key query(ID_i)*: Algorithm $\mathcal{B}_1$ prepares a list of tuples $\langle ID_i, x_i, D_i, pk_i, sk_i, c_i \rangle$ denoted by PSK^{list} and responds as follows:
 - Check the H_1^{list} , if $c_i = 0$, $\mathcal{B}_1$ executes the *Set-Secret-Value* algorithm to compute x_i , then calculates $D_i = (D_{i,1}, D_{i,2}) = (g_1^{z_i}, g_2^{z_i})$,

$sk_i = (sk_{i,1}, sk_{i,2}) = (D_{i,1}^{x_i}, D_{i,2}^{x_i})$, and $pk_i = (X_i, pk_{i,1}, pk_{i,2}) = (g^{x_i}, g_1^{x_i}, g_2^{x_i})$ by executing *Extract-Partial-Secret-Key*, *Set-Secret-Key*, and *Set-Public-Key* algorithms, respectively. Finally, $\mathcal{B}_1$ records item

$[ID_i, x_i, D_i, pk_i, sk_i, 0]$ into PSK^{list} and responds to $\mathcal{A}_1$ with pk_i .

- Otherwise, if $c_i = 1$, $\mathcal{B}_1$ executes the *Set-Secret-Value* algorithm to compute x_i , calculates $pk_i = (X_i, pk_{i,1}, pk_{i,2}) = (g^{x_i}, g_1^{x_i}, g_2^{x_i})$ by executing *Set-Public-Key* algorithm, and records item $[ID_i, x_i, *, pk_i, *, 1]$ into PSK^{list} and responds to $\mathcal{A}_1$ with pk_i .
- *Partial secret key query(ID_i)* If $c_i = 0$, $\mathcal{B}_1$ returns D_i associated with ID_i from PSK^{list} . Otherwise, if $c_i = 1$, $\mathcal{B}_1$ aborts and fails.
- *Secret key query(ID_i)* If $c_i = 0$, $\mathcal{B}_1$ returns sk_i associated with ID_i from PSK^{list} . Otherwise, if $c_i = 1$, $\mathcal{B}_1$ aborts and fails.
- *replace public key query(ID_i, pk'_i)* Suppose that $pk'_i = (X'_i, pk'_{i,1}, pk'_{i,2})$. Algorithm $\mathcal{B}_1$ verifies if $pk'_i = (X'_i, pk'_{i,1}, pk'_{i,2}) \in \mathbb{G}_1^*$ and $e(X'_i, g_1) = e(pk'_{i,1}, g)$ and $e(X'_i, g_2) = e(pk'_{i,2}, g)$. If these verifications pass, $\mathcal{B}_1$ replaces pk_i with pk'_i . Otherwise, $\mathcal{B}_1$ returns the symbol $\perp$ to $\mathcal{A}_1$.
- *Decryption query(ID_i, C_i)* Let $C_i = (C_{i,1}, C_{i,2}, C_{i,3}, C_{i,4})$. $\mathcal{B}_1$ searches on the PSK^{list} and reacts as follows:
 - If $c_i = 0$ and public key is not replaced, $\mathcal{B}_1$ executes *Decrypt*(C_i, sk_i) and returns M_i to $\mathcal{A}_1$.
 - Otherwise, for each item in H_3^{list} $\mathcal{B}_1$ responds as follows:
 - Calculate $M_i || r_{i,1} = C_{i,4} \oplus h_i$.
 - Verify if $C_{i,1} = g^{r_{i,1}}$ and $\frac{C_{i,3}}{M_i^{r_{i,1}}} = H_2(e(sk_{i,1}, C_{i,1}))$.
 - If both verifications pass, return M_i . Otherwise, return the symbol $\perp$.
- *Authorization query for $type_{\bowtie} (\bowtie = 1, 2, 4)(ID_i)$* : $\mathcal{B}_1$ searches on PSK^{list} with respect to the ID_i to get $sk_i = (g_1^{z_i}, g_2^{z_i})$ and responds to different types of authorizations as follows:
 - $\bowtie = 1$, $\mathcal{B}_1$ responds to the $\mathcal{A}_1$ with $td_{i,1} = g_1^{z_i x_i}$.
 - $\bowtie = 2$, $\mathcal{B}_1$ responds to the $\mathcal{A}_1$ with $td_{i,2} = H_2(e(g_1^{z_i}, C_{i,1}))$.
 - $\bowtie = 4$, $\mathcal{B}_1$ responds to the $\mathcal{A}_1$ with $td_{i,4} = (\frac{g^{y_i}}{H_2(e(g_1^{z_i}, C_{i,1}))}, e(g^{y_i}, C_{j,1}))$.

3. *Challenge* When $\mathcal{A}_1$ decides *Phase 1* is finished. It outputs identity ID_{ch} on which it wishes to be challenged. The algorithm $\mathcal{B}_1$ chooses $M^* \in_R \mathbb{G}_1^*$, then searches on PSK^{list} and reacts as follows:

- If $C_i = 0$, terminate with failure.
- Else, the following steps are performed:
 - choose $C_4 \in_R \{0, 1\}^{n_1+n_2}$ and set $C_2 = g^c$.
 - Generate $C^* = (C_1, C_2, C_3, C_4)$ and send to $\mathcal{A}_1$ as a challenge ciphertext.
- Observe that, the decryption of C^* is:

$$\begin{aligned} M^* || r &= C_4 \oplus H_3(e(sk_{i,2}, C_2)) \\ &= C_4 \oplus H_3(e(g_2^{bz_i x_i}, g^c)) \\ &= C_4 \oplus H_3(e(g^{abz_i x_i}, g^c)) \\ &= C_4 \oplus H_3(e(g, g)^{abc z_i x_i}). \end{aligned}$$

Phase 2 In this phase, the response of the challenger to $\mathcal{A}_1$ is similar of that one obtained in *Phase 1*. The following grounds are considered.

- ID_{ch} is not queried in the *Secret key queries*.
- If the public key associated with ID_{ch} is replaced, the ID_{ch} is not queried in the *Partial secret key queries*.
- If the public key of the user is replaced, the corresponding identity ID_i is not queried in the *Secret key queries*.
- (ID_{ch}, C^*) should not be queried in the *Decryption queries*.

4. *Guess* $\mathcal{A}_1$ returns M' for M^* . $\mathcal{B}_1$ picks item $[v_i, h_i] \in_R H_3^{list}$ and returns $h_i^{(zx)^{-1}} = e(g, g)^{abc}$ as the solution of BDH problem.

□

Claim If $\mathcal{B}_1$ holds in the simulation, then $\mathcal{A}_1$ is viewed similar to real attack. Thus, the $Pr[M^* = M'] \geq \epsilon_1$.

Proof H_1 -query responds as in the real attack. All replies to secret key query q_{sk} , partial secret key query q_{psk} , authorization query q_{aut} are valid. Then the probability of $\mathcal{A}_1$ is $Pr[M = M'] \geq \epsilon_1$. Then, we compute the probability of $\mathcal{B}_1$'s failure in the simulation as follows:

- The probability of $\mathcal{B}_1$ holds in *Phase 1* or *Phase 2* is equal to $\delta^{q_{sk}+q_{psk}+q_{aut}}$.
- The probability of $\mathcal{B}_1$ holds in *Challenge* phase is equal to $1 - \delta$.

- The combination of both gives the probability of $\mathcal{B}_1$ holds in the simulation is equal to $\delta^{q_{sk}+q_{psk}+q_{aut}}(1 - \delta)$.
- The maximum of this probability is equal to $\delta_{opt} = 1 - 1/(q_{sk} + q_{psk} + q_{aut} + 1)$.
- By using δ_{opt} , the probability of $\mathcal{B}_1$ holds is at least $1/e(q_{sk} + q_{psk} + q_{aut} + 1)$.

According to above analysis, the advantage of $\mathcal{B}_1$ is at least:

$$\epsilon_1(\lambda)/e(q_{sk} + q_{psk} + q_{aut} + 1) \quad (1)$$

In addition, the algorithm $\mathcal{B}_1$ is emulating the real attack environment to $\mathcal{A}_1$. Thus, $\mathcal{B}_1$ returns $e(g, g)^{abc}$ with probability at least:

$$\epsilon_1(\lambda)/q_{H_3}. \quad (2)$$

By combining Eqs. (1) and (2), we have

$$\epsilon_1(\lambda)/e(q_{sk} + q_{psk} + q_{aut} + 1) \cdot q_{H_3}.$$

□

Theorem 5.2 Suppose that H_1, H_2, H_3 are random oracles and assume that the BDH problem is hard. Therefore, our CL-PKC-AET is OW-CCA secure. Significantly, assume there is a *Type-2 adversary* $\mathcal{A}_2$ that has advantages $\epsilon_2(\lambda)$ against the CL-PKC-AET. Assume that $\mathcal{A}_2$ makes q_{pk} public key query, q_{sk} secret key query, q_{aut} authorization query for $type_{\mathbb{A}} (\bowtie = 1, 2, 4)$, q_{dec} decryption query, q_{H_2} hash query to H_2 , and q_{H_3} hash query to H_3 . Thus, we have an algorithm $\mathcal{B}_2$ which breaks BDH problem with advantage at least $\epsilon_2(\lambda)/e(q_{sk} + q_{aut} + 1) \cdot q_{H_3}$.

Proof Suppose that $\mathcal{B}_2$ is given as inputs the BDH parameters $\langle p, \mathbb{G}_1, \mathbb{G}_2, e \rangle$ and arbitrary instance $\langle g, g^a, g^b, g^c \rangle$ of the BDH problem, where g is a random generator of $\mathbb{G}_1$ and $a, b, c \in_R \mathbb{Z}_p^*$. Suppose that $e(g, g)^{abc} \in \mathbb{G}_2$ is the solution of BDH problem. Then, we demonstrate how an algorithm $\mathcal{B}_2$ obtains $e(g, g)^{abc}$ by reacting with $\mathcal{A}_1$ as follows:

1. *Setup* The algorithm $\mathcal{B}_2$ creates the public parameters $PubP = \langle p, \mathbb{G}_1, \mathbb{G}_2, e, g, g_1, g_2, H_1, H_2, H_3 \rangle$, where $g_1 = g^{s_1}$ and $g_2 = g^{s_2}$. Then, $\mathcal{B}_2$ gives $PubP$ and the master secret key $msk = (s_1, s_2)$ to $\mathcal{A}_2$.
2. *Phase 1* At any time $\mathcal{A}_2$ can make query to q_{pk} , q_{sk} , q_{aut} , q_{dec} , q_{H_2} , or q_{H_3} . To respond these queries $\mathcal{B}_2$ works as follows:
 - H_1 -query, H_2 -query, and H_3 -query are same as in *Phase 1* for proof of **Theorem 5.1**.
 - *Public key query*(ID_i): Algorithm $\mathcal{B}_2$ prepares a list of tuples

$\langle ID_i, x_i, D_i, pk_i, sk_i, c_i \rangle$ denoted by PSK^{list} and responds as follows:

- Check the H_1^{list} , if $c_i = 0$, $\mathcal{B}_2$ executes the *Set-Secret-Value* algorithm to compute x_i , then calculates $D_i = (D_{i,1}, D_{i,2}) = (g_1^{z_i}, g_2^{z_i})$, $sk_i = (sk_{i,1}, sk_{i,2}) = (D_{i,1}^{x_i}, D_{i,2}^{x_i})$, and $pk_i = (X_i, pk_{i,1}, pk_{i,2}) = (g^{x_i}, g_1^{x_i}, g_2^{x_i})$ by executing *Extract-Partial-Secret-Key*, *Set-Secret-Key*, and *Set-Public-Key* algorithms, respectively. Finally, $\mathcal{B}_2$ records item $[ID_i, x_i, D_i, pk_i, sk_i, 0]$ into PSK^{list} and responds to $\mathcal{A}_2$ with pk_i .
- Otherwise, if $c_i = 1$, $\mathcal{B}_2$ executes the *Extract-Partial-Secret-Key* algorithm to compute $D_i = (D_{i,1}, D_{i,2}) = (g_1^{z_i}, g_2^{z_i})$, then calculates $pk_i = (X_i, pk_{i,1}, pk_{i,2}) = (g^a, g_1^a, g_2^a)$ by executing *Set-Public-Key* algorithm, and records item $[ID_i, D_i, *, pk_i, *, 1]$ into PSK^{list} and responds to $\mathcal{A}_2$ with pk_i .
- *Secret key query*(ID_i): If $c_i = 0$, $\mathcal{B}_2$ returns sk_i associated with ID_i from PSK^{list} . Otherwise, if $c_i = 1$, $\mathcal{B}_2$ aborts and fails.
- *Decryption query*(ID_i, C_i): Let $C_i = (C_{i,1}, C_{i,2}, C_{i,3}, C_{i,4})$. $\mathcal{B}_2$ searches on the PSK^{list} and reacts as follows:
 - If $c_i = 0$, $\mathcal{B}_2$ executes *Decrypt*(C_i, sk_i) and returns M_i to $\mathcal{A}_2$.
 - Otherwise, for each item in H_3^{list} $\mathcal{B}_2$ responds as follows:
 - Calculate $M_i || r_{i,1} = C_{i,4} \oplus h_i$.
 - Verify if $C_{i,1} = g^{r_{i,1}}$ and $\frac{C_{i,3}}{M_i^{r_{i,1}}} = H_2(e(sk_{i,1}, C_{i,1}))$.
 - If both verifications pass, return M_i . Otherwise, return the symbol $\perp$.
- *Authorization query for type $_{\bowtie}$* ($\bowtie = 1, 2, 4$)(ID_i): $\mathcal{B}_2$ searches on PSK^{list} with respect to the ID_i and responds to different types of authorizations as follows:

- If $c_i = 0$, $\mathcal{B}_2$ responds as follows:

- $\bowtie = 1$, $\mathcal{B}_2$ responds to the $\mathcal{A}_2$ with $td_{i,1} = g_1^{z_i x_i}$.
- $\bowtie = 2$, $\mathcal{B}_2$ responds to the $\mathcal{A}_2$ with $td_{i,2} = H_2(e(g_1^{z_i x_i}, C_{i,1}))$.
- $\bowtie = 4$, $\mathcal{B}_2$ responds to the $\mathcal{A}_2$ with $td_{i,4} = (\frac{g^{y_i}}{H_2(e(g_1^{z_i x_i}, C_{i,1}))}), e(g^{y_i}, C_{j,1}))$.

- Else, if $c_i = 1$, $\mathcal{B}_2$ responds as follows:

- $\bowtie = 1$, $\mathcal{B}_2$ responds to the $\mathcal{A}_2$ with $td_{i,1} = g_1^{a z_i}$.
- $\bowtie = 2$, $\mathcal{B}_2$ responds to the $\mathcal{A}_2$ with $td_{i,2} = H_2(e(g_1^{a z_i}, C_{i,1}))$.
- $\bowtie = 4$, $\mathcal{B}_2$ responds to the $\mathcal{A}_2$ with $td_{i,4} = (\frac{g^{y_i}}{H_2(e(g_1^{a z_i}, C_{i,1}))}), e(g^{y_i}, C_{j,1}))$.

3. *Challenge* When $\mathcal{A}_2$ determines *Phase 1* is finished, it outputs identity ID_{ch} on which it wishes to be challenged. The algorithm $\mathcal{B}_2$ selects $M^* \in_R \mathbb{G}_1^*$, then searches on PSK^{list} and reacts as follows:

- If $C_i = 0$, terminate with failure.
- Else, the following steps are performed:
 - choose $C_4 \in_R \{0, 1\}^{n_1+n_2}$ and set $C_2 = g^c$.
 - Generate $C^* = (C_1, C_2, C_3, C_4)$ and send to $\mathcal{A}_2$ as a challenge ciphertext.
- the decryption of C^* will then be:

$$\begin{aligned} M^* || r &= C_4 \oplus H_3(e(sk_{i,2}, C_2)) \\ &= C_4 \oplus H_3(e(g_2^{b z_i}, g^c)) \\ &= C_4 \oplus H_3(e(g^{abs_2 z_i}, g^c)) \\ &= C_4 \oplus H_3(e(g, g)^{abcs_2 z_i}). \end{aligned}$$

4. *Phase 2* In this phase, the response of the challenger to $\mathcal{A}_2$ is similar of that one obtained in *Phase 1* on grounds that:

- ID_{ch} is not queried in the *Secret key query*.
- (ID_{ch}, C^*) is not queried in the *Decryption query*.

5. *Guess* $\mathcal{A}_2$ returns M' for M^* . $\mathcal{B}_2$ picks item $[v_i, h_i] \in_R H_3^{list}$ and returns $h_i^{(z_{s_2})^{-1}} = e(g, g)^{abc}$ as the solution of BDH problem.

□

Claim If $\mathcal{B}_2$ holds in the simulation, then $\mathcal{A}_2$ is viewed similar to real attack. Thus, the $Pr[M^* = M'] \geq \epsilon_1$.

Proof H_1 -query responds as in the real attack. All replies to secret key query q_{sk} and authorization query q_{aut} are valid. Then the probability of $\mathcal{A}_2$ is $Pr[M^* = M'] \geq \epsilon_2$. Then, we compute the probability of $\mathcal{B}_2$'s failure in the simulation as follows:

- The probability of $\mathcal{B}_2$ holds in *Phase 1* or *Phase 2* is equal to $\delta^{q_{sk}+q_{aut}}$.

- The probability of $\mathcal{B}_2$ holds in *Challenge* phase is equal to $1 - \delta$.
- The combination of both gives the probability of $\mathcal{B}_2$ holds in the simulation is equal to $\delta^{q_{sk}+q_{aut}}(1 - \delta)$.
- The maximum of this probability is equal to $\delta_{opt} = 1 - 1/(q_{sk} + q_{aut} + 1)$.
- By using δ_{opt} , the probability of $\mathcal{B}_2$ holds is at least $1/e(q_{sk} + q_{aut} + 1)$.

According to above analysis, the advantage of $\mathcal{B}_2$ is at least:

$$\epsilon_2(\lambda)/e(q_{sk} + q_{aut} + 1) \quad (3)$$

In addition, the algorithm $\mathcal{B}_2$ is emulating the real attack environment to $\mathcal{A}_2$. Thus, $\mathcal{B}_2$ returns $e(g, g)^{abc}$ with probability at least:

$$\epsilon_2(\lambda)/q_{H_3}. \quad (4)$$

By combining Eqs. (3) and (4), we have

$$\epsilon_2(\lambda)/e(q_{sk} + q_{aut} + 1) \cdot q_{H_3}.$$

□

Theorem 5.3 Suppose that H_1, H_2, H_3 are random oracles and assume that the BDH problem is hard. Therefore, our CL-PKC-AET is INC-CCA secure. Significantly, assume there is a *Type-3 adversary* $\mathcal{A}_3$ that has advantages $\epsilon_3(\lambda)$ against the CL-PKC-AET. Assume that $\mathcal{A}_3$ makes q_{pk} public key query, q_{sk} secret key query, q_{psk} partial secret key query, q_{aut} authorization query for $type_{\mathbb{A}} (\mathbb{A} = 1, 2, 4)$, q_{rp} replace public key query, q_{dec} decryption query, q_{H_2} hash query to H_2 , and q_{H_3} hash query to H_3 . Thus, we have an algorithm $\mathcal{B}_3$ which breaks BDH problem with advantage at least $\epsilon_3(\lambda)/e(q_{sk} + q_{psk} + q_{aut} + 1) \cdot q_{H_3}$.

Proof Suppose that $\mathcal{B}_3$ is given as inputs the BDH parameters $\langle p, \mathbb{G}_1, \mathbb{G}_2, e \rangle$ and arbitrary instance $\langle g, g^a, g^b, g^c \rangle$ of the BDH problem, where g is a random generator of $\mathbb{G}_1$ and $a, b, c \in_R \mathbb{Z}_p^*$. Suppose that $e(g, g)^{abc} \in \mathbb{G}_2$ is the solution of BDH problem. Then, we demonstrate how an algorithm $\mathcal{B}_3$ obtains $e(g, g)^{abc}$ by reacting with $\mathcal{A}_3$ as follows:

1. *Setup* The algorithm $\mathcal{B}_3$ gives $\mathcal{A}_3$ the public parameters $PubP = \langle p, \mathbb{G}_1, \mathbb{G}_2, e, g, g_1, g_2, H_1, H_2, H_3 \rangle$, where $g_1 = g_2 = g^a$.
2. *Phase 1* Similar to the *Phase 1* in the proof of **Theorem 5.1**.
3. *Challenge* When $\mathcal{A}_3$ decides *Phase 1* is finished, it outputs two plaintext M_0, M_1 and the identity ID_{ch} on which it wishes to be challenged. The algorithm $\mathcal{B}_3$ chooses

$b \in_R \{0, 1\}$, and goes ahead to search on PSK^{list} and reacts as follows:

- If $C_i = 0$, terminate with failure.
- Else, the following steps are performed:
 - choose $C_4 \in_R \{0, 1\}^{n_1+n_2}$ and set $C_2 = g^c$.
 - Generate $C^* = (C_1, C_2, C_3, C_4)$ and send to $\mathcal{A}_1$ as a challenge ciphertext.
- Observe that, the decryption of C^* is:

$$\begin{aligned} M_b || r &= C_4 \oplus H_3(e(sk_{i,2}, C_2)) \\ &= C_4 \oplus H_3(e(g_2^{bz_i x_i}, g^c)) \\ &= C_4 \oplus H_3(e(g^{abz_i x_i}, g^c)) \\ &= C_4 \oplus H_3(e(g, g)^{abc z_i x_i}). \end{aligned}$$

4. *Phase 2* In this phase, the response of the challenger to $\mathcal{A}_3$ is similar of that one obtained in *Phase 1* on grounds that:

- ID_{ch} is not queried in the *Secret key query*.
- If the public key associated with ID_{ch} is replaced, the ID_{ch} is not queried in the *Partial secret key query*.
- If the public key of the user is replaced, the corresponding identity ID_i should not be queried in the *Secret key query*.
- (ID_{ch}, C^*) is not queried in the *Decryption query*.
- ID_{ch} is not queried in the *Authorization query*.

5. *Guess* $\mathcal{A}_3$ returns b' for b . $\mathcal{B}_3$ picks item $[v_i, h_i] \in_R H_3^{list}$ and returns $h_i^{(zx)^{-1}} = e(g, g)^{abc}$ as the solution of BDH problem.

□

Claim If $\mathcal{B}_3$ holds in the simulation, then $\mathcal{A}_3$ is viewed similar to real attack. Thus, the $|Pr[b' = b] - \frac{1}{2}| \geq \epsilon_3$.

Proof H_1 -query responds as in the real attack. All replies to secret key query q_{sk} , partial secret key query q_{psk} , authorization query q_{aut} are valid. Then the probability of $\mathcal{A}_3$ is $|Pr[b' = b] - \frac{1}{2}| \geq \epsilon_3$. Then, we compute the probability of $\mathcal{B}_3$'s failure in the simulation as follows:

- The probability of $\mathcal{B}_3$ holds in *Phase 1* or *Phase 2* is equal to $\delta^{q_{sk}+q_{psk}+q_{aut}}$.
- The probability of $\mathcal{B}_3$ holds in *Challenge* phase is equal to $1 - \delta$.
- The combination of both gives the probability of $\mathcal{B}_3$ holds in the simulation is equal to $\delta^{q_{sk}+q_{psk}+q_{aut}}(1 - \delta)$.

- The maximum of this probability is equal to $\delta_{opt} = 1 - 1/(q_{sk} + q_{psk} + q_{aut} + 1)$.
- By using δ_{opt} , the probability of $\mathcal{B}_3$ holds is at least $1/e(q_{sk} + q_{psk} + q_{aut} + 1)$.

According to above analysis, the advantage of $\mathcal{B}_3$ is at least:

$$\epsilon_3(\lambda)/e(q_{sk} + q_{psk} + q_{aut} + 1) \quad (5)$$

In addition, the algorithm $\mathcal{B}_3$ is emulating the real attack environment to $\mathcal{A}_3$. Thus, $\mathcal{B}_3$ returns $e(g, g)^{abc}$ with probability at least:

$$\epsilon_3(\lambda)/q_{H_3}. \quad (6)$$

By combining Eqs. (5) and (6), we have

$$\epsilon_1(\lambda)/e(q_{sk} + q_{psk} + q_{aut} + 1) \cdot q_{H_3}.$$

□

Theorem 5.4 Suppose that H_1, H_2, H_3 are random oracles and assume that the BDH problem is hard. Therefore, our CL-PKC-AET is IND-CCA secure. Significantly, assume there is a *Type-4 adversary* $\mathcal{A}_4$ that has advantages $\epsilon_4(\lambda)$ against the CL-PKC-AET. Assume that $\mathcal{A}_4$ makes q_{pk} public key query, q_{sk} secret key query, q_{aut} authorization query for type-4 ($\bowtie = 1, 2, 4$), q_{dec} decryption query, q_{H_2} hash query to H_2 , and q_{H_3} hash query to H_3 . Thus, we have an algorithm $\mathcal{B}_4$ which breaks BDH problem with advantage at least $\epsilon_4(\lambda)/e(q_{sk} + q_{aut} + 1) \cdot q_{H_3}$.

Proof Suppose that $\mathcal{B}_4$ is given as inputs the BDH parameters $\langle p, \mathbb{G}_1, \mathbb{G}_2, e \rangle$ and arbitrary instance $\langle g, g^a, g^b, g^c \rangle$ of the BDH problem, where g is a random generator of $\mathbb{G}_1$ and $a, b, c \in_R \mathbb{Z}_p^*$. Suppose that $e(g, g)^{abc} \in \mathbb{G}_2$ is the solution of BDH problem. Then, we demonstrate how an algorithm $\mathcal{B}_4$ obtains $e(g, g)^{abc}$ by reacting with $\mathcal{A}_4$ as follows:

1. *Setup* The algorithm $\mathcal{B}_4$ creates the public parameters $PubP = \langle p, \mathbb{G}_1, \mathbb{G}_2, e, g, g_1, g_2, H_1, H_2, H_3 \rangle$, where $g_1 = g^{s_1}$ and $g_2 = g^{s_2}$. Then, $\mathcal{B}_4$ gives $PubP$ and the master secret key $msk = (s_1, s_2)$ to $\mathcal{A}_4$.
2. *Phase 1* Same as the *Phase 1* in the proof of **Theorem 5.2**.
3. *Challenge* When $\mathcal{A}_4$ decides *Phase 1* is finished, it outputs two plaintext M_0, M_1 and the identity ID_{ch} on which it wishes to be challenged. The algorithm $\mathcal{B}_4$ chooses $b \in_R \{0, 1\}$, then searches on PSK^{list} and reacts as follows:

- If $C_i = 0$, terminate with failure.
- Else, the following steps are performed:

- choose $C_4 \in_R \{0, 1\}^{n_1+n_2}$ and set $C_2 = g^c$.
- Generate $C^* = (C_1, C_2, C_3, C_4)$ and send to $\mathcal{A}_2$ as a challenge ciphertext.

- the decryption of C^* will then be:

$$\begin{aligned} M^* || r &= C_4 \oplus H_3(e(sk_{i,2}, C_2)) \\ &= C_4 \oplus H_3(e(g_2^{bz_i}, g^c)) \\ &= C_4 \oplus H_3(e(g^{abs_2 z_i}, g^c)) \\ &= C_4 \oplus H_3(e(g, g)^{abcs_2 z_i}). \end{aligned}$$

4. *Phase 2* In this phase, the response of the challenger to $\mathcal{A}_2$ is similar of that one obtained in *Phase 1* on grounds that:

- ID_{ch} is not queried in the *Secret key query*.
- (ID_{ch}, C^*) is not queried in the *Decryption query*.
- ID_{ch} is not queried in the *Authorization query*.

5. *Guess* $\mathcal{A}_4$ returns b' for b . $\mathcal{B}_4$ picks item $[v_i, h_i] \in_R H_3^{list}$ and returns $h_i^{(cx)^{-1}} = e(g, g)^{abc}$ as the solution of BDH problem.

□

Claim If $\mathcal{B}_4$ holds in the simulation, then $\mathcal{A}_4$ is viewed similar to real attack. Thus, the $|Pr[b' = b] - \frac{1}{2}| \geq \epsilon_4$.

Proof H_1 -query responds as in the real attack. All replies to secret key query q_{sk} and authorization query q_{aut} are valid. Then the probability of $\mathcal{A}_4$ is $|Pr[b' = b] - \frac{1}{2}| \geq \epsilon_4$. Then, we compute the probability of $\mathcal{B}_4$'s failure in the simulation as follows:

- The probability of $\mathcal{B}_4$ holds in *Phase 1* or *Phase 2* is equal to $\delta^{q_{sk}+q_{aut}}$.
- The probability of $\mathcal{B}_4$ holds in *Challenge* phase is equal to $1 - \delta$.
- The combination of both gives the probability of $\mathcal{B}_4$ holds in the simulation is equal to $\delta^{q_{sk}+q_{aut}}(1 - \delta)$.
- The maximum of this probability is equal to $\delta_{opt} = 1 - 1/(q_{sk} + q_{aut} + 1)$.
- By using δ_{opt} , the probability of $\mathcal{B}_4$ holds is at least $1/e(q_{sk} + q_{aut} + 1)$.

According to above analysis, the advantage of $\mathcal{B}_4$ is at least:

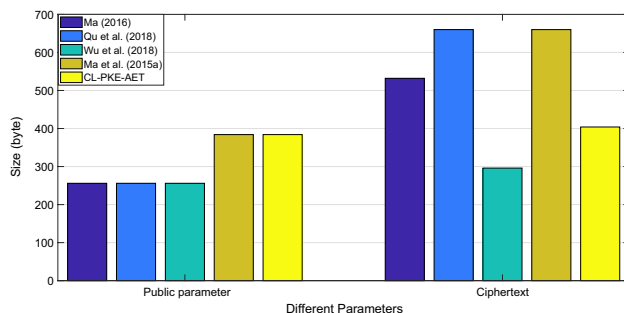
$$\epsilon_4(\lambda)/e(q_{sk} + q_{aut} + 1) \quad (7)$$

In addition, the algorithm $\mathcal{B}_4$ is emulating the real attack environment to $\mathcal{A}_4$. Thus, $\mathcal{B}_4$ returns $e(g, g)^{abc}$ with probability at least:

Table 1 Performance comparison of different schemes

Schemes	$ PK $	$ CT $	C_E	C_D	C_A	C_T
Ma (2016)	2G	$4G + \mathbb{Z}_p$	$6e$	$2p + 2e$	–	$4p$
Qu et al. (2018)	2G	$5G + \mathbb{Z}_p$	$5e$	$2p + 2e$	–	$4p$
Wu et al. (2018)	2G	$2G + 2\mathbb{Z}_p$	$2e$	$2p$	–	$2p + 2e$
Ma et al. (2015a)-1	3G	$5G + \mathbb{Z}_p$	$6e$	$5e$	0	$2p + 2e$
Ma et al. (2015a)-2	3G	$5G + \mathbb{Z}_p$	$6e$	$5e$	$2e$	$2p + 2e$
Ma et al. (2015a)-3	3G	$5G + \mathbb{Z}_p$	$6e$	$5e$	$2p + 4e$	$2p + 2e$
Ma et al. (2015a)-4	3G	$5G + \mathbb{Z}_p$	$6e$	$5e$	$1e$	$2p + 2e$
CL-PKC-AET-1	3G	$3G + \mathbb{Z}_p$	$5e$	$2p + 2e$	0	$4p$
CL-PKC-AET-2	3G	$3G + \mathbb{Z}_p$	$5e$	$2p + 2e$	$2e$	$2p$
CL-PKC-AET-3	3G	$3G + \mathbb{Z}_p$	$5e$	$2p + 2e$	$1p$	$3p$
CL-PKC-AET-4	3G	$3G + \mathbb{Z}_p$	$5e$	$2p + 2e$	$4p + 2e$	$2p$

$|PK|$, the size of the public key; $|CT|$ the size of the ciphertext; C_E , the computation complexity of encryption; C_D , the computation complexity of decryption; C_A , the computation complexity of authorization; C_T , the computation complexity of test algorithms; e , an exponentiation operation; p , pairing operation; –, this property is not considered in the corresponding scheme; CL-PKC-AET _{$\bowtie$} and Ma et al. (2015a) _{$\bowtie$} ($\bowtie = 1, 2, 3, 4$): *type _{$\bowtie$}* authorization for our scheme and the scheme in (Ma et al. 2015a)

**Fig. 4** Comparison of communication cost

$$\epsilon_4(\lambda)/q_{H_3}. \quad (8)$$

By combining Eqs. (7) and (8), we have

$$\epsilon_4(\lambda)/e(q_{sk} + q_{aut} + 1) \cdot q_{H_3}.$$

□

6 Performance analysis

In terms of communication cost, computation cost, and property, the proposed CL-PKC-AET scheme, and other alternatives are compared in this section.

6.1 Communication cost

As we shown in Table 1 and Fig. 4, in terms of the size of public key, our scheme has the same size as (Ma et al. 2015a)

and it's bigger than others. In terms of the ciphertext's size, our scheme has the smallest ciphertext size, except (Wu et al. 2018) has smaller size than ours.

6.2 Computation cost

As we shown in Table 1 and Fig. 5, for the Encryption algorithm, our scheme has less computational cost than (Ma et al. 2015a; Ma 2016). It's same as (Qu et al. 2018) and larger than (Wu et al. 2018). In terms of the Decryption algorithm, our scheme has larger computational cost than (Wu et al. 2018; Ma et al. 2015a). It's same with (Ma 2016; Qu et al. 2018). In terms of the Authorization algorithm, the type-2, type-3, and type-4 authorizations of our scheme have larger computational cost than other schemes except type-3 in (Ma et al. 2015a). Moreover, the computational cost of type-1 in our scheme is equal to zero. Finally, for the Test algorithm, the type-2 and type-4 authorizations of our scheme have smaller computational cost than other schemes.

6.3 Property comparison

In terms of keyword search, equality test, certificate management problems, key escrow problem, authorization equality test, and security, our scheme and the related works are compared in Table 2. Obviously, our scheme is fulfilled all properties.

6.4 Comparison with other schemes in IIoT

Recently, Gope et al. (2019) and Wang et al. (2018) also proposed such schemes to tackle the security and privacy concerns in the IIoT environments. However, compared

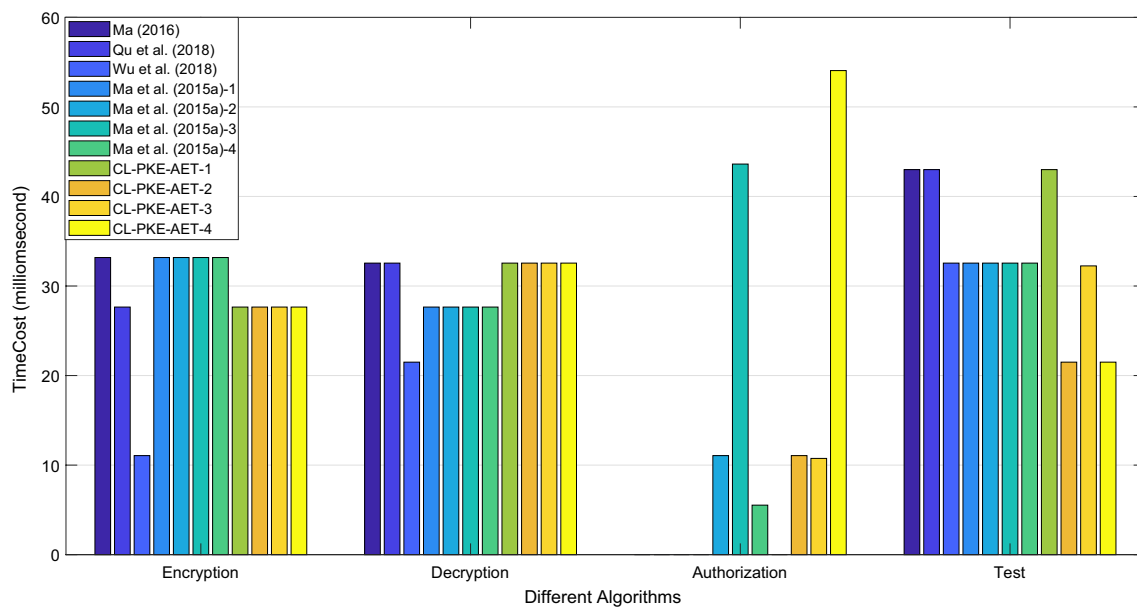


Fig. 5 Comparison of the time-cost for Encryption, Decryption, Authorization, and Test algorithms

Table 2 Property comparison of different schemes

Schemes	KS	ET	CMP	KEP	AET	S_{Aut}	S_{W-Aut}	Cryptosystem
Ma (2016)	✓	✓	×	✓	×	OW-ID-CCA	–	IBC-based
Qu et al. (2018)	✓	✓	×	×	×	OW-CCA	–	CLC-based
Wu et al. (2018)	✓	✓	×	✓	×	OW-ID-CCA	–	IBC-based
Ma et al. (2015a)	✓	✓	✓	✓	✓	OW-CCA	IND-CCA	PKI-based
CL-PKC-AET	✓	✓	×	×	✓	OW-CCA	IND-CCA	CLC-based

KS, keyword search; ET, equality test; CMP, certificate management problems; KEP, key escrow problem; AET, authorized equality test; S_{Aut} , security with consideration of the authorization; S_{W-Aut} , security without consideration of the authorization; $mark$, represents that the property we choose is fulfilled by the scheme; $\times$, represents that the property we choose is not fulfilled by the scheme; $-$, this property is not considered in the corresponding scheme

our solution with them, our proposed scheme provides confidentiality, which the data collected by the sensors node is encrypted before outsourced to the cloud server. In addition, our proposed scheme offers to the authorized users to access and search on the data stored on the cloud server with different types of authorizations. However, the schemes proposed by Gope et al. (2019) and Wang Wang et al. (2018) only provide authentication protocols in which only the user with a trusted device has the right to access the industrial wireless sensor network (IWSN). But this user cannot search the data stored in IWSN or cloud server.

6.5 Implementation

The implementation of our scheme, combined with the schemes in (Ma et al. 2015a; Ma 2016; Wu et al. 2018; Qu et al. 2018) (shown in Figs. 4 and 5), is based on the cpabe

and the Pairing-Based Cryptography (PBC) library (Lynn 2013). We notably adopted an Intel(R) Core(TM) i7-7700 CPU @3.60 GHz @3.60 GHz and 8 GB RAM. We used the Windows 10 operating system and VC++ 6.0 for our experiments to run on. We implemented the 160-bit elliptic curve group and constructed it on the super-singular curve $z^2 = x^3 + x$ over a 512-bit finite field, to attain a 128-bit security level. For the cost of computation and communication overhead, we had milliseconds (ms) and Kilobyte (KB), respectively. In our experimental simulation, the time of the pairing operation and the exponentiation operation are 10.749 ms and 5.53 ms, respectively. Besides, we have the size of each element in $\mathbb{Z}_p$, $\mathbb{G}_1$, and $\mathbb{G}_2$ are 20 bytes, 128 bytes, 128 bytes, respectively.

An actualization that our scheme supports four types of authorization to ensure equality tests in ciphertexts without the loss in efficiency resulted after a concise comparison with (Ma 2016; Qu et al. 2018; Wu et al. 2018). Compared

with (Ma et al. 2015a), our proposed scheme is constructed on the certificateless cryptography which eradicates the key escrow problem featured in the IBE-ET schemes as well as the management of the public key certificates featured in the PKI-ET schemes and some other issues such as revocation and verification issues. Considering the restricted memory that IIoT devices contain, in comparison with other schemes, we are convinced our solution is preferable.

7 Conclusion

In the paper, an efficient certificateless public key cryptography supporting authorized equivalence test is presented. The authorization mechanism in our proposed scheme is flexible such that the testing capability is delegated by the user to the cloud server with different types of authorizations. Moreover, by utilizing the random oracle model, the security proof of our proposed scheme is reduced to the BDH assumption. The theoretical analysis, together with the experiment simulation demonstrates that our proposed scheme is practical. To our knowledge, this is the first CL-PKC-AET scheme, which is especially suitable for ciphertext searching in the cloud assisted IIoT environment. The future work consists of constructing CL-PKC scheme with verifiable and authorized equality test. Namely, the delegating user can check the equivalence test results received from the cloud server.

Acknowledgements This work was supported in part by the 13th Five-Year Plan of National Cryptography Development Fund for Cryptographic Theory of China under Grant MMJJ20170204, in part by the Fundamental Research Funds for the Central Universities under Grant ZYGX2016J091, the Guangxi Colleges and Universities Key Laboratory of Cloud Computing and Complex Systems, and in part by the Natural Science Foundation of China under Grants U1401257, 61472064 and 61602096.

References

- Abdalla M, Bellare M, Catalano D, Kiltz E, Kohno T, Lange T, Malone-Lee J, Neven G, Paillier P, Shi H (2005) Searchable encryption revisited: consistency properties, relation to anonymous IBE, and extensions. *Crypto* 3621:205–222 (Springer)
- Al-Riyami SS, Paterson KG (2003) Certificateless public key cryptography. *Asiacrypt* 2894:452–473 (Springer)
- Anagnostopoulos I, Zeadally S, Exposito E (2016) Handling big data: research challenges and future directions. *J Supercomput* 72(4):1494–1516
- Atzori L, Iera A, Morabito G (2010) The internet of things: a survey. *Comput Netw* 54(15):2787–2805
- Baek J, Safavi-Naini R, Susilo W (2008) Public key encryption with keyword search revisited. *Comput Sci Appl ICCSA* 2008:1249–1259
- Boneh D, Di Crescenzo G, Ostrovsky R, Persiano G (2004) Public key encryption with keyword search. In: *International conference on the theory and applications of cryptographic techniques*, Springer, pp 506–522
- Elhabob R, Zhao Y, Sella I, Xiong H (2019) Efficient certificateless public key cryptography with equality test for internet of vehicles. *IEEE Access*, pp 1–1
- Esposito C, Castiglione A, Martini B, Choo KKR (2016) Cloud manufacturing: security, privacy, and forensic concerns. *IEEE Cloud Comput* 3(4):16–22
- Gope P, Das AK, Kumar N, Cheng Y (2019) Lightweight and physically secure anonymous mutual authentication protocol for real-time data access in industrial wireless sensor networks. *IEEE transactions on industrial informatics*
- Hassan A, Eltayieb N, Elhabob R, Li F (2018) An efficient certificateless user authentication and key exchange protocol for client-server environment. *J Ambient Intell Hum Comput* 9(6):1713–1727
- Huang K, Tso R, Chen YC, Rahman SMM, Almogren A, Alamri A (2015) Pke-aet: Public key encryption with authorized equality test. *Comput J* 58(10):2686–2697
- Kim S, Lee I (2018) Iot device security based on proxy re-encryption. *J Ambient Intell Hum Comput* 9(4):1267–1273
- Lee HT, Ling S, Seo JH, Wang H (2016) Semi-generic construction of public key encryption and identity-based encryption with equality test. *Inf Sci* 373:419–440
- Lynn B (2013) The stanford pairing based crypto library. In: *Privacy preservation scheme for multicasting communications in smart buildings of the smart grid*, pp 324
- Ma S (2016) Identity-based encryption with outsourced equality test in cloud computing. *Inf Sci* 328:389–402
- Ma S, Huang Q, Zhang M, Yang B (2015a) Efficient public key encryption with equality test supporting flexible authorization. *IEEE Trans Inf Forensics Secur* 10(3):458–470
- Ma S, Zhang M, Huang Q, Yang B (2015b) Public key encryption with delegated equality test in a multi-user setting. *Comput J* 58(4):986–1002
- Molano JIR, Lovelle JMC, Montenegro CE, Granados JJR, Crespo RG (2018) Metamodel for integration of internet of things, social networks, the cloud and industry 4.0. *J Ambient Intell Hum Comput* 2018:1–15
- Noroozi M, Eslami Z (2019) Public-key encryption with keyword search: a generic construction secure against online and offline keyword guessing attacks. *J Ambient Intell Hum Comput* 2019:1–12
- Premkamal PK, Pasupuleti SK, Alphonse P (2018) A new verifiable outsourced ciphertext-policy attribute based encryption for big data privacy and access control in cloud. *J Ambient Intell Hum Comput* 2018:1–15
- Qu H, Yan Z, Lin XJ, Zhang Q, Sun L (2018) Certificateless public key encryption with equality test. *Inf Sci* 462:76–92
- Singh S, Sharma PK, Moon SY, Park JH (2017) Advanced lightweight encryption algorithms for iot devices: survey, challenges and solutions. *J Ambient Intell Hum Comput* 2017:1–18
- Tang Q (2011) Towards public key encryption scheme supporting equality test with fine-grained authorization. In: *Australasian Conference on Information Security and Privacy*, Springer, pp 389–406
- Tang Q (2012a) Public key encryption schemes supporting equality test with authorisation of different granularity. *Int J Appl Cryptogr* 2(4):304–321
- Tang Q (2012b) Public key encryption supporting plaintext equality test and user-specified authorization. *Secur Commun Netw* 5(12):1351–1362
- Truong HL, Dustdar S (2015) Principles for engineering iot cloud systems. *IEEE Cloud Comput* 2(2):68–76

- Wang D, Li W, Wang P (2018) Measuring two-factor authentication schemes for real-time data access in industrial wireless sensor networks. *IEEE Trans Ind Inf* 14(9):4081–4092
- Wu L, Zhang Y, Choo KKR, He D (2017) Efficient and secure identity-based encryption scheme with equality test in cloud computing. *Future Gener Comput Syst* 73:22–31
- Wu L, Zhang Y, Choo KKR, He D (2018) Efficient identity-based encryption scheme with equality test in smart city. *IEEE Trans Sustain Comput* 3(1):44–55
- Xu Y, Wang M, Zhong H, Cui J, Liu L, Franqueira VN (2017) Verifiable public key encryption scheme with equality test in 5G networks. *IEEE Access*
- Yang G, Tan CH, Huang Q, Wong DS (2010) Probabilistic public key encryption with equality test. In: *CryptographersTrack at the RSA Conference*, Springer, pp 119–131

Publisher's Note Springer Nature remains neutral with regard to jurisdictional claims in published maps and institutional affiliations.

Terms and Conditions

Springer Nature journal content, brought to you courtesy of Springer Nature Customer Service Center GmbH (“Springer Nature”).

Springer Nature supports a reasonable amount of sharing of research papers by authors, subscribers and authorised users (“Users”), for small-scale personal, non-commercial use provided that all copyright, trade and service marks and other proprietary notices are maintained. By accessing, sharing, receiving or otherwise using the Springer Nature journal content you agree to these terms of use (“Terms”). For these purposes, Springer Nature considers academic use (by researchers and students) to be non-commercial.

These Terms are supplementary and will apply in addition to any applicable website terms and conditions, a relevant site licence or a personal subscription. These Terms will prevail over any conflict or ambiguity with regards to the relevant terms, a site licence or a personal subscription (to the extent of the conflict or ambiguity only). For Creative Commons-licensed articles, the terms of the Creative Commons license used will apply.

We collect and use personal data to provide access to the Springer Nature journal content. We may also use these personal data internally within ResearchGate and Springer Nature and as agreed share it, in an anonymised way, for purposes of tracking, analysis and reporting. We will not otherwise disclose your personal data outside the ResearchGate or the Springer Nature group of companies unless we have your permission as detailed in the Privacy Policy.

While Users may use the Springer Nature journal content for small scale, personal non-commercial use, it is important to note that Users may not:

1. use such content for the purpose of providing other users with access on a regular or large scale basis or as a means to circumvent access control;
2. use such content where to do so would be considered a criminal or statutory offence in any jurisdiction, or gives rise to civil liability, or is otherwise unlawful;
3. falsely or misleadingly imply or suggest endorsement, approval, sponsorship, or association unless explicitly agreed to by Springer Nature in writing;
4. use bots or other automated methods to access the content or redirect messages
5. override any security feature or exclusionary protocol; or
6. share the content in order to create substitute for Springer Nature products or services or a systematic database of Springer Nature journal content.

In line with the restriction against commercial use, Springer Nature does not permit the creation of a product or service that creates revenue, royalties, rent or income from our content or its inclusion as part of a paid for service or for other commercial gain. Springer Nature journal content cannot be used for inter-library loans and librarians may not upload Springer Nature journal content on a large scale into their, or any other, institutional repository.

These terms of use are reviewed regularly and may be amended at any time. Springer Nature is not obligated to publish any information or content on this website and may remove it or features or functionality at our sole discretion, at any time with or without notice. Springer Nature may revoke this licence to you at any time and remove access to any copies of the Springer Nature journal content which have been saved.

To the fullest extent permitted by law, Springer Nature makes no warranties, representations or guarantees to Users, either express or implied with respect to the Springer nature journal content and all parties disclaim and waive any implied warranties or warranties imposed by law, including merchantability or fitness for any particular purpose.

Please note that these rights do not automatically extend to content, data or other material published by Springer Nature that may be licensed from third parties.

If you would like to use or distribute our Springer Nature journal content to a wider audience or on a regular basis or in any other manner not expressly permitted by these Terms, please contact Springer Nature at

onlineservice@springernature.com